



ADVANCED DATABASE SYSTEMS

WEEK 12

Parallelism in Database Systems



Outline

- Parallel Database Systems
 - Parallel Architectures
 - Data Placement
 - Query Processing
 - Load Balancing and Fault-tolerance
 - Database Clusters

Recommended Reading:

Chapter 8 in Principles of Distributed Database Systems.

M. Tamer Özsu · Patrick Valduriez. Springer ISBN 978-3-030-26252-5 2020



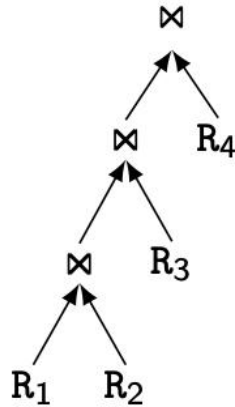
Parallel Query Optimization

- The objective is to select the “best” parallel execution plan for a query using the following components
- Search strategy
 - Dynamic programming for small search space
 - Randomized for large search space
- Search space
 - Models alternative execution plans as operator trees
 - Different tree shapes
- Cost model (abstraction of execution system)
 - Physical schema info. (partitioning, indexes, etc.)
 - Statistics and cost functions

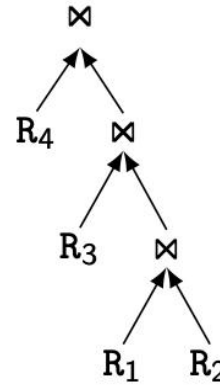


Operator Trees

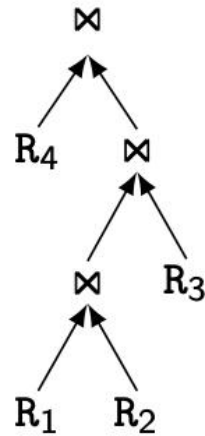
Left-deep



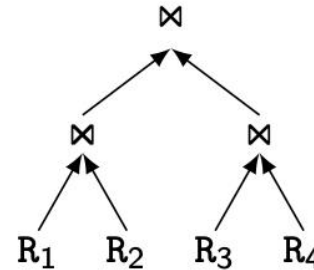
Right-deep



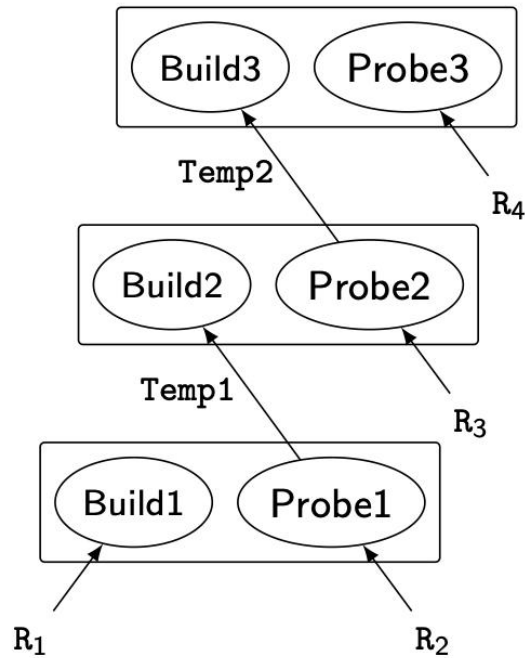
Zig-zag



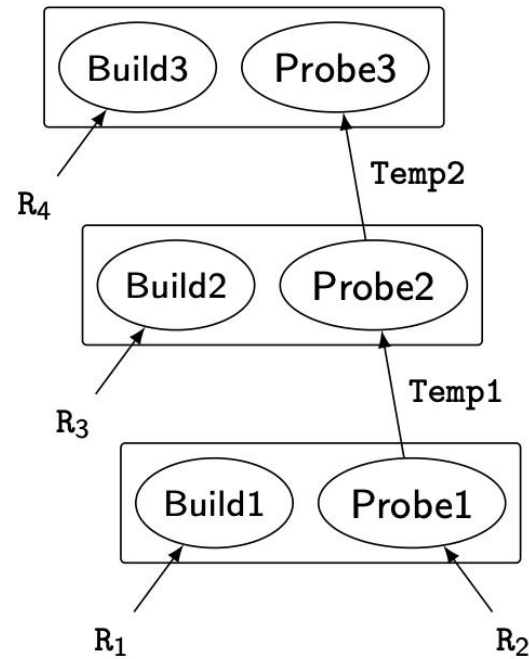
Bushy



Equivalent Hash-Join Trees with Different Scheduling



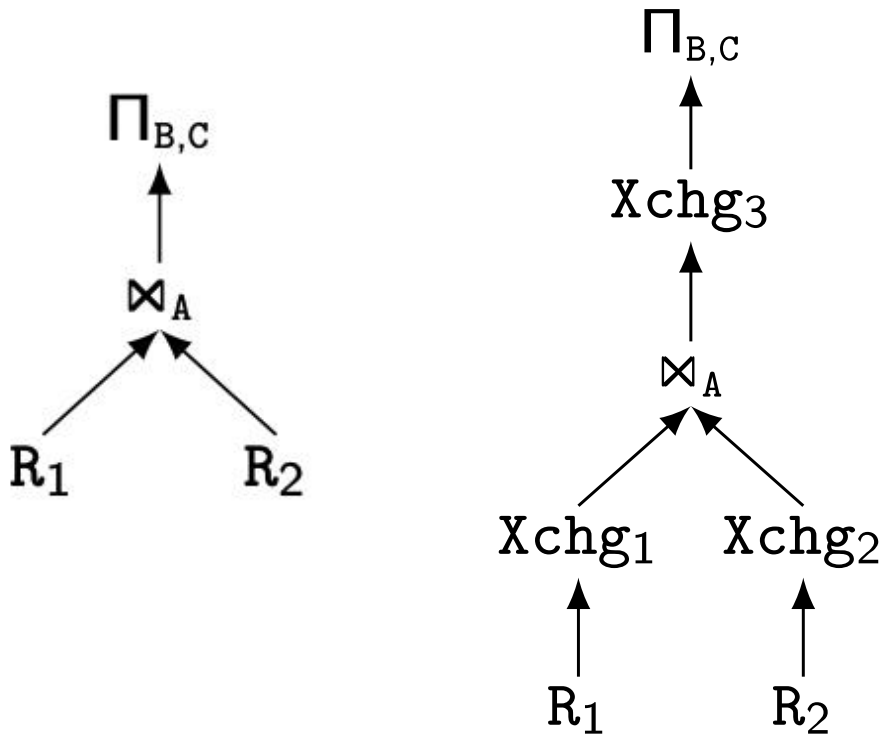
No pipeline



Pipeline of R_2 , Temp₁ and Temp₂



Operator Tree with Exchange Operator



$Xchg_1$: partition by $h(A)$

$Xchg_2$: partition by $h(A)$

$Xchg_3$: partition by $h(B, C)$



Cost Model

- Total time
 - Similar to distributed query optimization
 - Adds all CPU, I/O and **com.** Costs
- Response time
 - More involved as it must take into account pipelining
 - Schedule plan p in phases ph

$$respTime(p) = \sum_{ph \in p} (\max_{Op \in ph} (respTime(Op) + pipeDelay(Op)) + storeDelay(ph))$$

where

- ▶ $pipeDelay(Op)$ is the time necessary for the operator Op to deliver the first result tuples
- ▶ $storeDelay(ph)$ is the time necessary to store the result of phase ph



Outline

- Parallel Database Systems
 - Parallel Architectures
 - Data Placement
 - Query Processing
 - Load Balancing and Fault-tolerance
 - Database Clusters

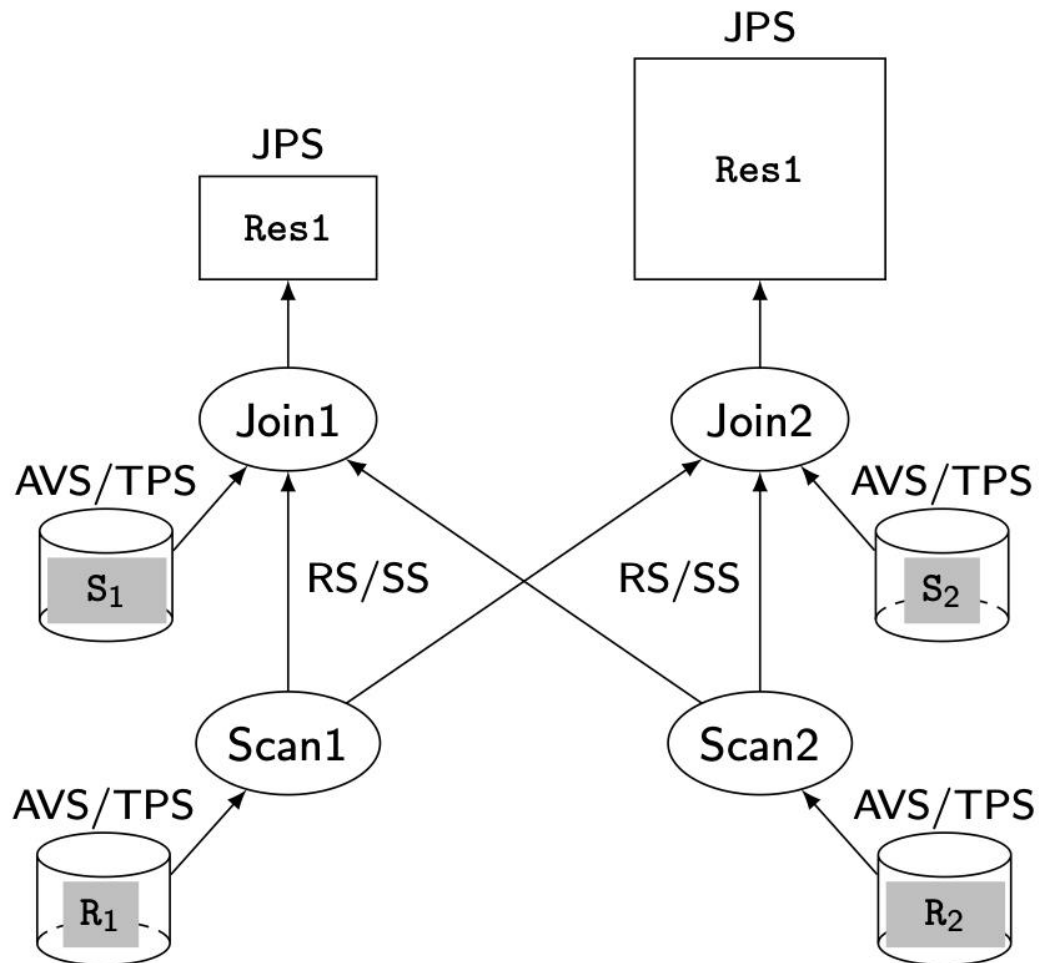


Load Balancing

- ❑ Initialization
- ❑ Interference
- ❑ Problems arise for intra-operator parallelism with *skewed* data distributions
 - ❑ Attribute data skew (AVS)
 - ❑ Tuple placement skew (TPS)
 - ❑ Selectivity skew (SS)
 - ❑ Redistribution skew (RS)
 - ❑ Join product skew (JPS)
- ❑ Solutions
 - ❑ Sophisticated parallel algorithms that deal with skew
 - ❑ Dynamic processor allocation (at execution time)

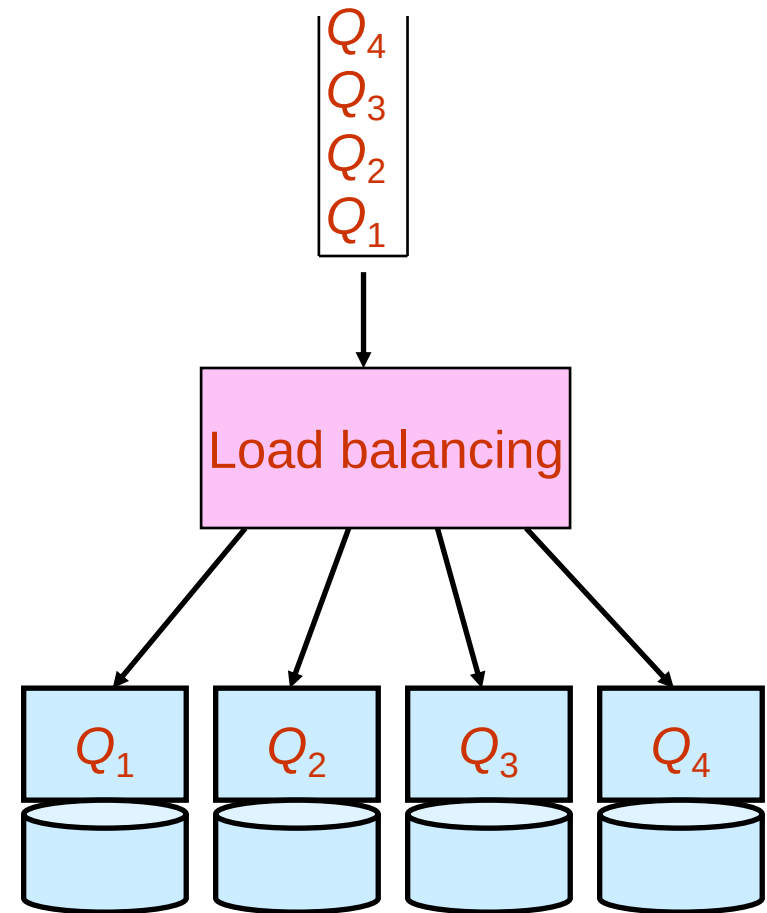


Data Skew Example



Load Balancing

- ❑ Choose the node to execute Q
 - ❑ Round robin
 - ❑ The least loaded
 - ▶ Need to get load information
- ❑ Failover
 - ❑ In case a node N fails, N 's queries are taken over by another node
 - ▶ Requires a copy of N 's data
- ❑ In case of interference
 - ❑ Data of an overloaded node are replicated to another node



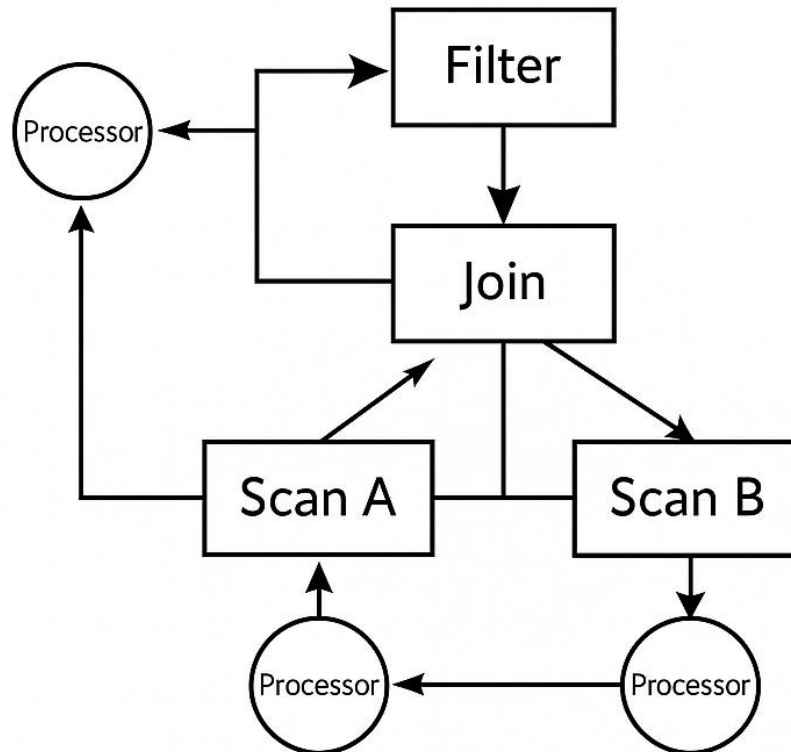
Dynamic Processing (DP) Execution Model

- A query is decomposed into self-contained units of sequential processing, each of which can be carried out by any processor
- Intuitively, a processor can migrate horizontally (intraoperator parallelism) and vertically (interoperator parallelism) along the query operators
- The parallel execution plan as produced by the optimizer includes operator scheduling constraints that express a partial order among the operators of the query



Dynamic Processing (DP) Execution Model

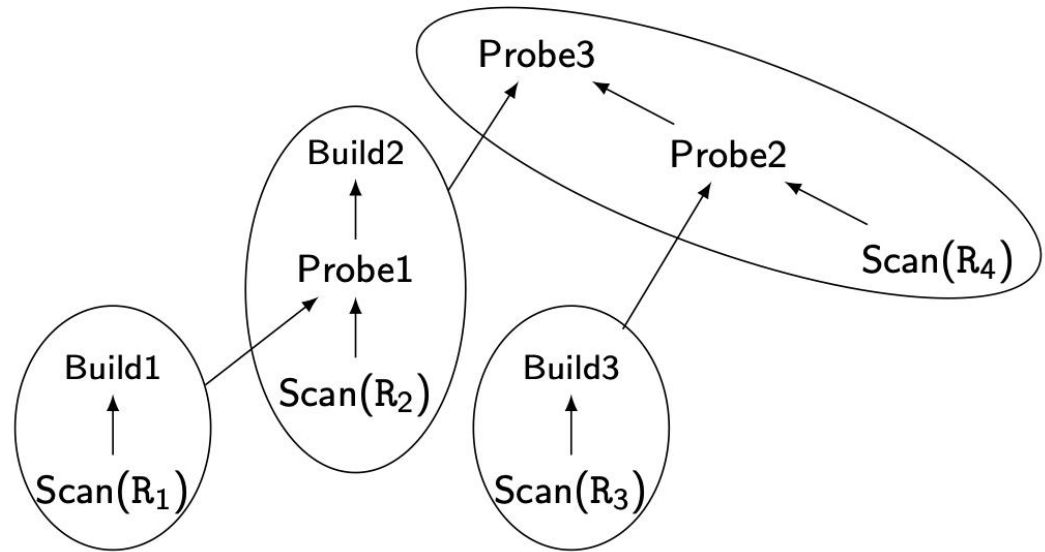
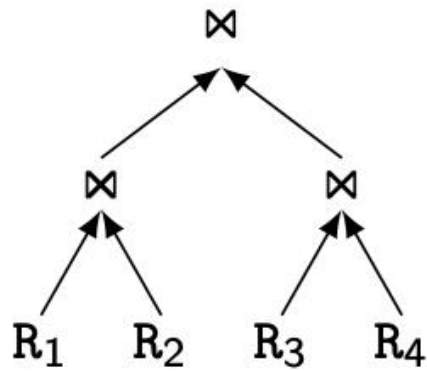
A query is decomposed into self-contained units of sequential processing, each of which can be carried out by any *processor*



Intuitively, a processor can migrate horizontally (*intraoperator parallelism*) and vertically (*interoperator parallelism*) along the query operators

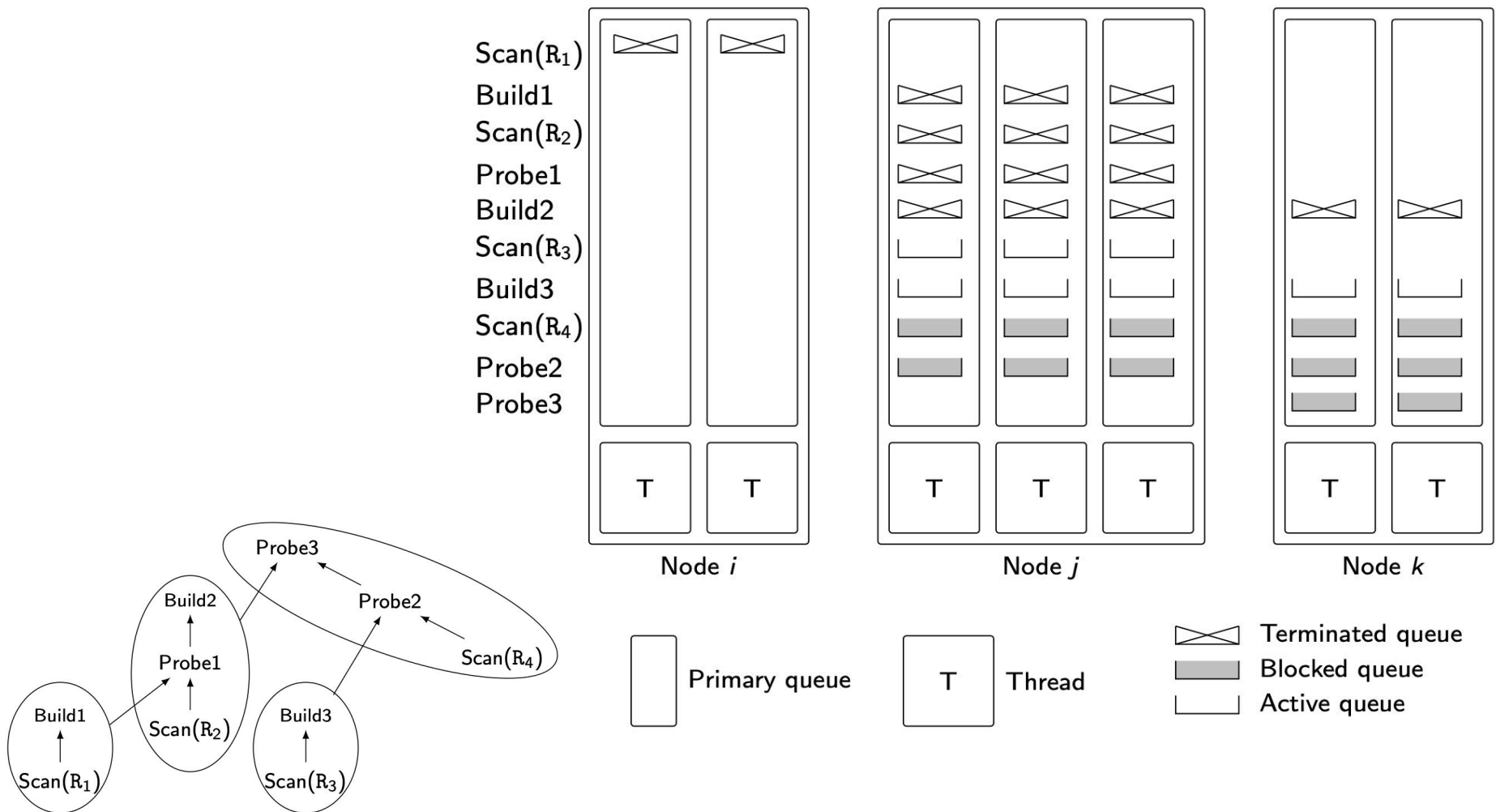
The parallel execution plan as produced by the optimizer

DP Example



Join tree with 4 pipelined chains

DP Example – snapshot of execution



Replication and Failover

□ Replication

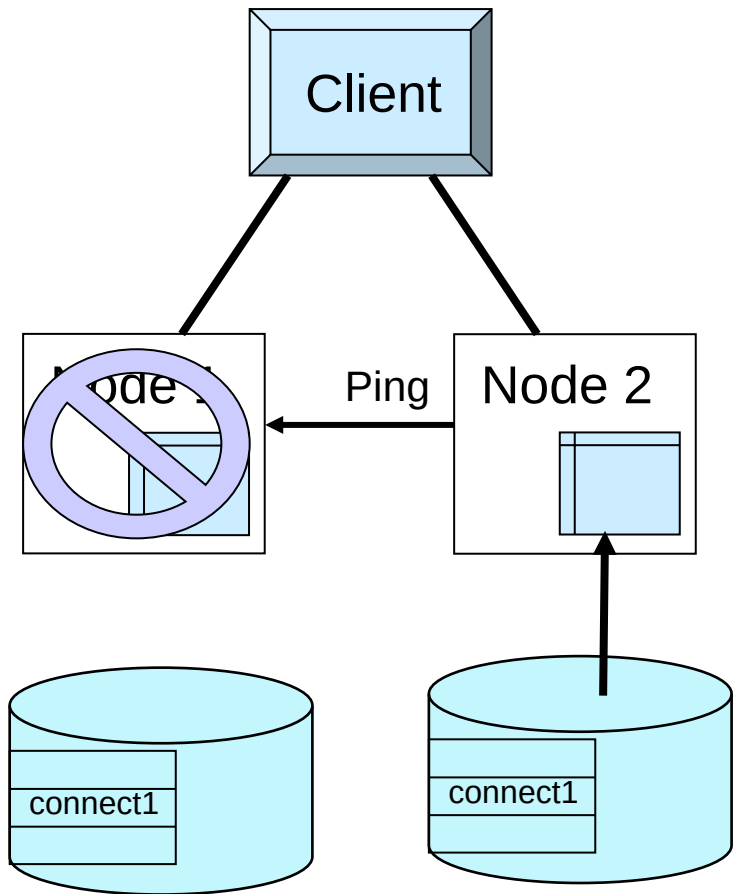
- The basis for fault-tolerance and availability

□ Failover

- On a node failure, another node detects and recovers the node's tasks

=>

Savepoints for long queries



Outline

- Parallel Database Systems
 - Parallel Architectures
 - Data Placement
 - Query Processing
 - Load Balancing and Fault-tolerance
 - Database Clusters

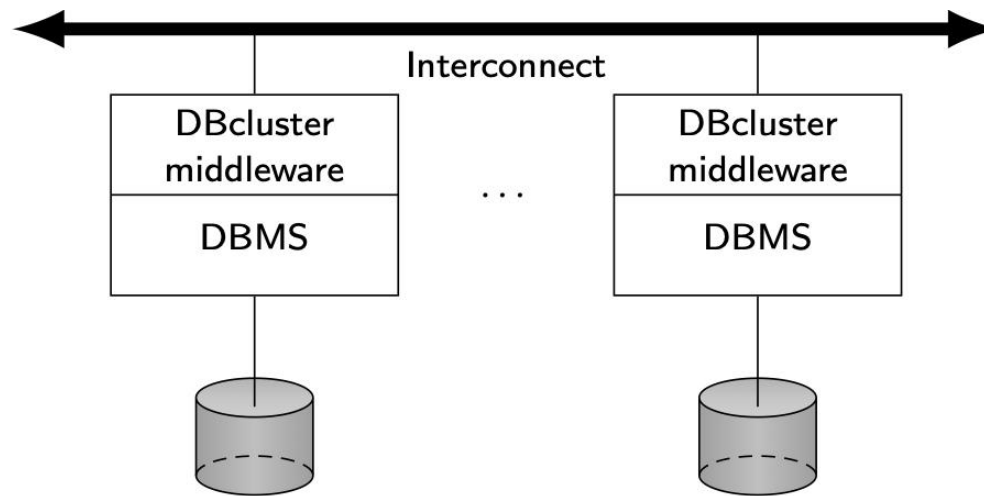


Motivations

- ❑ Database cluster = cluster of autonomous databases, each managed by an off-the-shelf DBMS
- ❑ Major difference with a parallel DBMS is the use of a “black-box” DBMS at each node
 - ❑ In general, same DBMS
- ❑ Since the DBMS source code is not necessarily available and cannot be changed to be “cluster-aware,” parallel data management capabilities must be implemented via middleware
- ❑ Examples: MySQL or PostgreSQL clusters



Database Cluster Architecture



- Middleware has several software layers
 - Transaction load balancer, replication manager, query processor, and fault-tolerance manager

Replication

- ❑ The fast interconnect and communication system can be exploited to support one-copy serializability
- ❑ Example with Preventive Replication
 - ❑ Uses FIFO reliable multicast (simple and efficient)
 - ❑ Principle
 - ▶ Each incoming transaction T has a chronological timestamp $ts(T) = C$, and is multicast to all other nodes where there is a copy.
 - ▶ At each node, a time delay is introduced before starting the execution of T . This delay corresponds to the upper bound of the time needed to multicast a message
 - ▶ When the delay expires, all transactions that may have committed before C are guaranteed to be received and executed before T , following the timestamp order (i.e., total order)

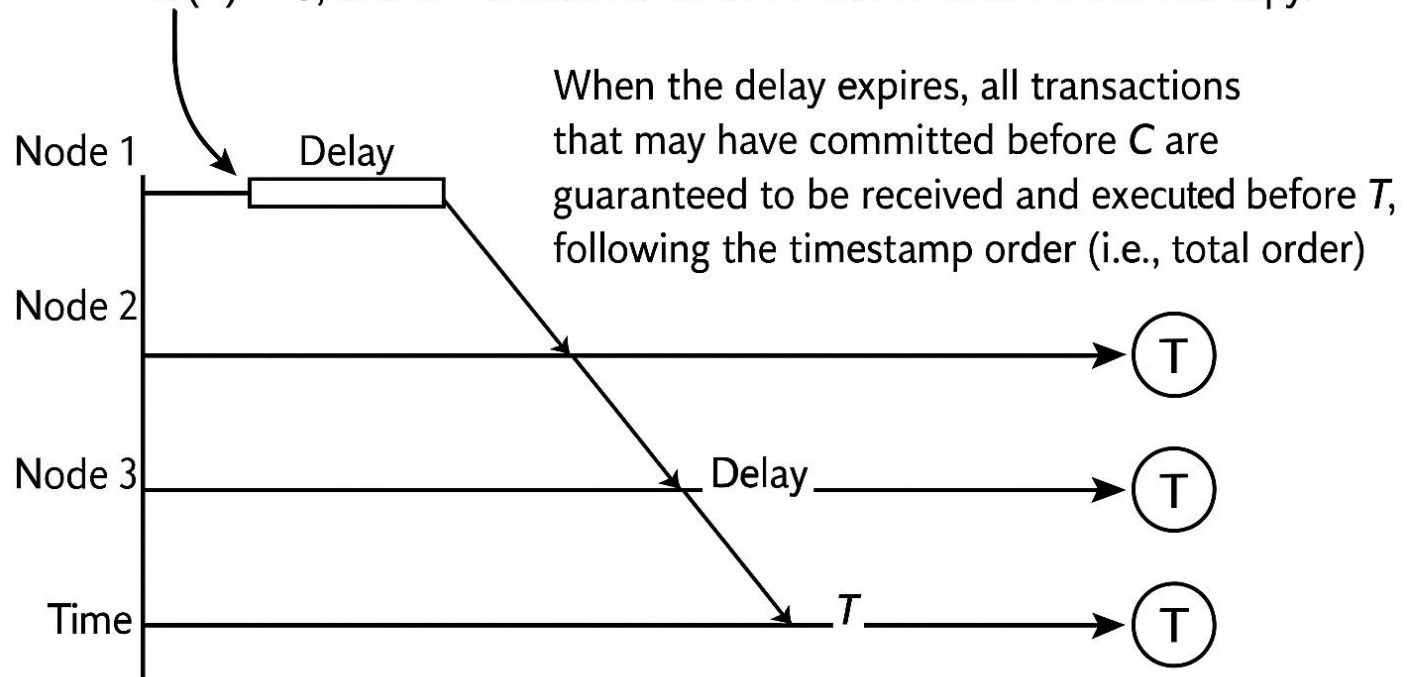


Replication

Example with Preventive Replication

Uses FIFO reliable multicast (simple and efficient)

Each incoming transaction T has a chronological timestamp $ts(T) = C$, and is multicast to all other nodes where there is a copy.



Load Balancing

- Query load balancing is easy
 - Since all copies are mutually consistent, any node that stores a copy of the data, e.g., the least loaded node, can be chosen at runtime by a conventional load balancing strategy
- Transaction load balancing
 - The total cost of transaction execution at all nodes may be high
 - By relaxing consistency, lazy replication can better reduce transaction execution cost and thus increase performance of both queries and transactions



Query Processing

- Interquery parallelism is naturally obtained as a result of load balancing and replication
 - Useful to increase the throughput of transaction-oriented applications and, to some extent, to reduce the response time of transactions and queries
- Intraquery parallelism is essential to further reduce response times of OLAP queries
 - More difficult than data partitioning because of black-box DBMSs
 - 2 solutions
 - ▶ Physical partitioning
 - ▶ Virtual partitioning



Physical Partitioning

- Similar to data partitioning in distributed databases except that the objective is to increase intraquery parallelism, not locality of reference
- Thus, depending on the query and relation sizes, the degree of partitioning should be much finer
- Under uniform data distribution, yields good intraquery parallelism and outperforms interquery parallelism
- However, it is static and thus very sensitive to data skew conditions and the variation of query patterns that may require periodic repartitioning



Virtual Partitioning

- Uses full replication (each relation is replicated at each node).
- Simple virtual partitioning (SVP)
 - Virtual partitions are dynamically produced for each query and intraquery parallelism is obtained by sending subqueries to different virtual partitions
 - To produce the different subqueries, the query processor adds predicates to the incoming query in order to restrict access to a subset of a relation, i.e., a virtual partition
 - Then, each DBMS that receives a subquery is forced to process a different subset of data items
 - Finally, the partitioned result needs to be combined by an aggregate query



SVP Example

- Consider

```
SELECT  PNO, AVG (DUR)
FROM    WORKS
WHERE    SUM (DUR) > 200
GROUP BY PNO
```

- A generic subquery on a virtual partition is obtained by adding to Q's where clause the predicate

```
AND PNO >= 'P1' AND PNO < 'P2'
```



SVP Assessment

- ❑ Great flexibility for node allocation during query processing since any node can be chosen for executing a subquery
- ❑ Not all kinds of queries can benefit from SVP
 - ❑ Only queries without subqueries that access a fact table are ok
 - ❑ Queries with a subquery must be converted to have no subquery
 - ❑ Any other queries cannot benefit
- ❑ Limitations
 - ❑ Determining the best virtual partitioning attributes and value ranges can be difficult
 - ❑ Dependent on the underlying DBMS query capabilities
- ❑ Solution: Adaptive Virtual Partitioning

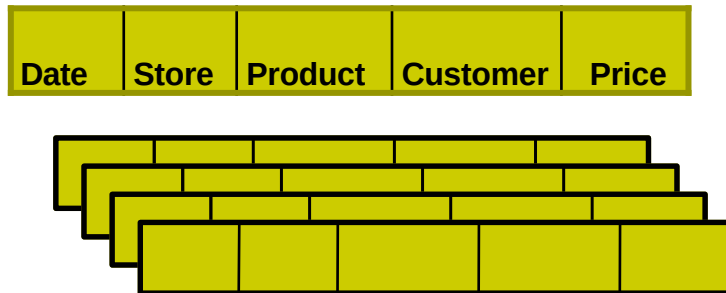


COLUMN-ORIENTED ARCHITECTURES



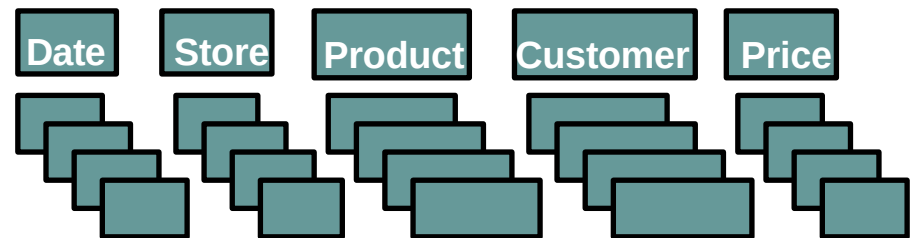
What is a column-store?

row-store



- + easy to add/modify a record
- might read in unnecessary data

column-store



- + only need to read in relevant data
- tuple writes require multiple accesses

=> suitable for read-mostly, read-intensive, large data repositories

The Analytical Bottleneck in Row-Oriented Layouts

Date	Customer	Product	Store
2023-10-26	C123	P555	S10
2023-10-23	C123	P555	S11
2023-10-26	C123	P555	S10
2023-10-21	C133	P560	S10
2023-10-24	C142	P562	S10
2023-10-27	C123	P560	S10

row store

Date	Customer	Product	Store
2023-10-26	C123	P555	S10
2023-10-23	C123	P555	S11
2023-10-26	C123	P555	S10
2023-10-21	C133	P560	S10
2023-10-24	C142	P562	S10
2023-10-27	C123	P560	S10

column store

The OLTP Pattern

Frequent single-row reads and updates.
Needs contiguous row access for single-entity integrity.

The OLAP Pattern

Aggregating specific fields across millions of rows (e.g., SUM(price) WHERE region='US').

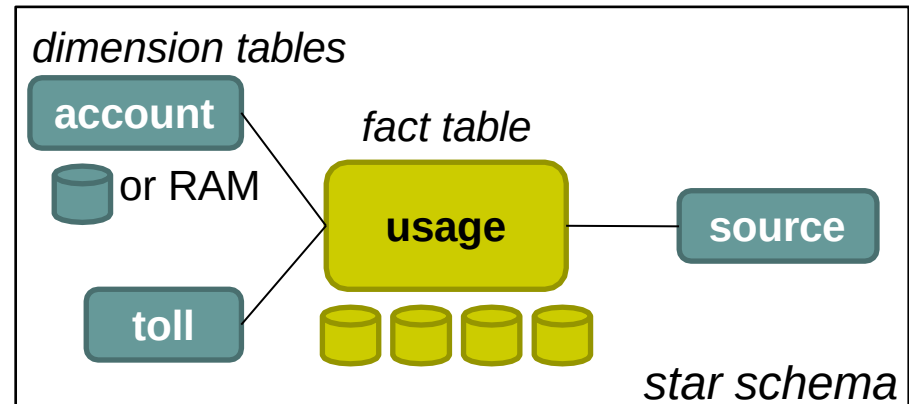
The I/O Penalty (Telco Example)

A typical warehouse query calculating **average toll prices** forces a **row-store** to read **212** columns per record just to access the 2 needed columns, wasting immense I/O bandwidth.

Telco Data Warehousing Example

1 Typical DW installation

1 Real-world example



QUERY 2

```
SELECT account.account_number,  
       sum (usage.toll_airtime),  
       sum (usage.toll_price)  
FROM usage, toll, source, account  
WHERE usage.toll_id = toll.toll_id  
AND usage.source_id = source.source_id  
AND usage.account_id = account.account_id  
AND toll.type_ind in ('AE', 'AA')  
AND usage.toll_price > 0  
AND source.type != 'CIBER'  
AND toll.rating_method = 'IS'  
AND usage.invoice_date = 20051013  
GROUP BY account.account_number
```

Column-store

Query 1 2.06
Query 2 2.20
Query 3 0.09
Query 4 5.24
Query 5 2.88

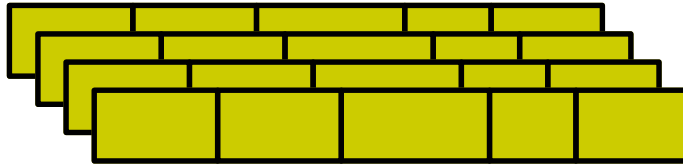
Row-store

300
300
300
300
300

Why? Three main factors (next slides)

Telco example explained (1/3): *read efficiency*

row store

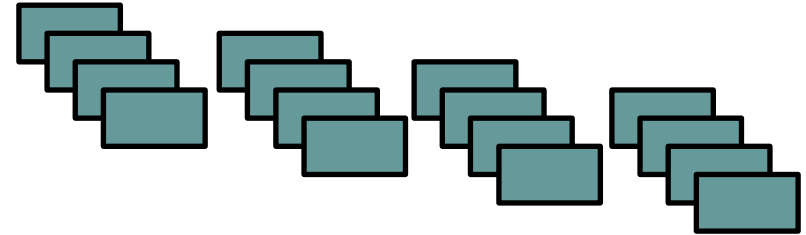


read pages containing entire rows

one row = 212 columns!

is this typical? (it depends)

column store



read only columns needed

in this example: 7 columns

caveats:

- “select * ” not any faster
- clever disk prefetching
- clever tuple reconstruction

The Physics of Physical Storage Layouts

NSM (N-ary Storage Model)

ID	Name	Age	Salary
0100	Alice	30	75000

Serializes every field of a logical record into a single contiguous block on disk.

Optimal for retrieving

`SELECT * WHERE id = X.`

DSM (Decomposition Storage Model)

Virtual ID	Name	Age	Salary
...	...	...	...
0100	Alice	30	75000
0962	Bob	45	92000
1000	Charlie	28	68000

Each attribute (column) is stored in its own isolated contiguous block.

The I/O Mathematics

On a table with 50 columns, an analytical query reading 3 columns in a column-store performs roughly 6% of the physical disk I/O required by a traditional row-store.

Telco example explained (2/3): *compression efficiency*

- 1 Columns compress better than rows
 - 1 Typical row-store compression ratio 1 : 3
 - 1 Column-store 1 : 10
- 1 Why?
 - 1 Rows contain values from different domains
=> more entropy, difficult to dense-pack
 - 1 Columns exhibit significantly less entropy
 - 1 Examples:

Male, Female, Female, Female, Male
1998, 1998, 1999, 1999, 1999, 2000
 - 1 Caveat: CPU cost (use lightweight compression)

The Physics of Columnar Compression

Context: Columns contain homogeneous data types, resulting in low information entropy. Less data on disk equals reduced I/O. The CPU operates directly on compressed data, making compression a query accelerator, not just a storage saver.

Run-Length Encoding (RLE)

Collapses sequences of identical values. Optimal for sorted data.

USA, USA, USA → [USA, 3]

Dictionary Encoding

Replaces heavy string values with lightweight integer IDs.

'New York' → 1, 'London' → 2

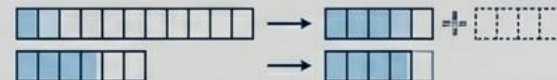
Delta Encoding

Stores only the difference between consecutive values. Crucial for timestamps and sequential IDs.

1000, 1005, 1008 → 1000, +5, +3

Bit Packing

Uses the strict minimum number of bits required to represent a restricted range of integers, stripping away padding.



Telco example explained (3/3): *sorting & indexing efficiency*

- 1 Compression and dense-packing free up space
 - 1 Use multiple overlapping column collections
 - 1 Sorted columns compress better
 - 1 Range queries are faster
 - 1 Use sparse clustered indexes

What about heavily-indexed row-stores?
(works well for single column access,
cross-column joins become increasingly expensive)

Hardware Realities Forced an Evolutionary Shift

Date	Customer	Product	Store
2023-10-26	C123	P555	\$10
2023-10-23	C123	P555	\$11
2023-10-26	C123	P555	\$10
2023-10-21	C133	P560	\$10
2023-10-24	C142	P562	\$10
2023-10-27	C123	P560	\$10

row store

Date	Customer	Product	Store
2023-10-26	C123	P555	\$10
2023-10-23	C123	P555	\$11
2023-10-26	C123	P555	\$10
2023-10-21	C133	P560	\$10
2023-10-24	C142	P562	\$10
2023-10-27	C123	P560	\$10

column store

70s 77s 1985 1990 1999 2005 Today

1985: Conceptualization

Copeland & Khoshafian publish "A Decomposition Storage Model (DSM)", proposing data organization by column. Initially deemed too storage-heavy.

1999: The Memory Wall

MonetDB identifies memory access as the primary bottleneck, using vertical fragmentation to optimize cache-to-RAM communication.

2005: The Turning Point

C-Store formalizes the read-optimized column DBMS, proving that trading abundant CPU cycles for scarce disk bandwidth yields massive performance gains.

Bridging the Gap with Cache-Conscious Design

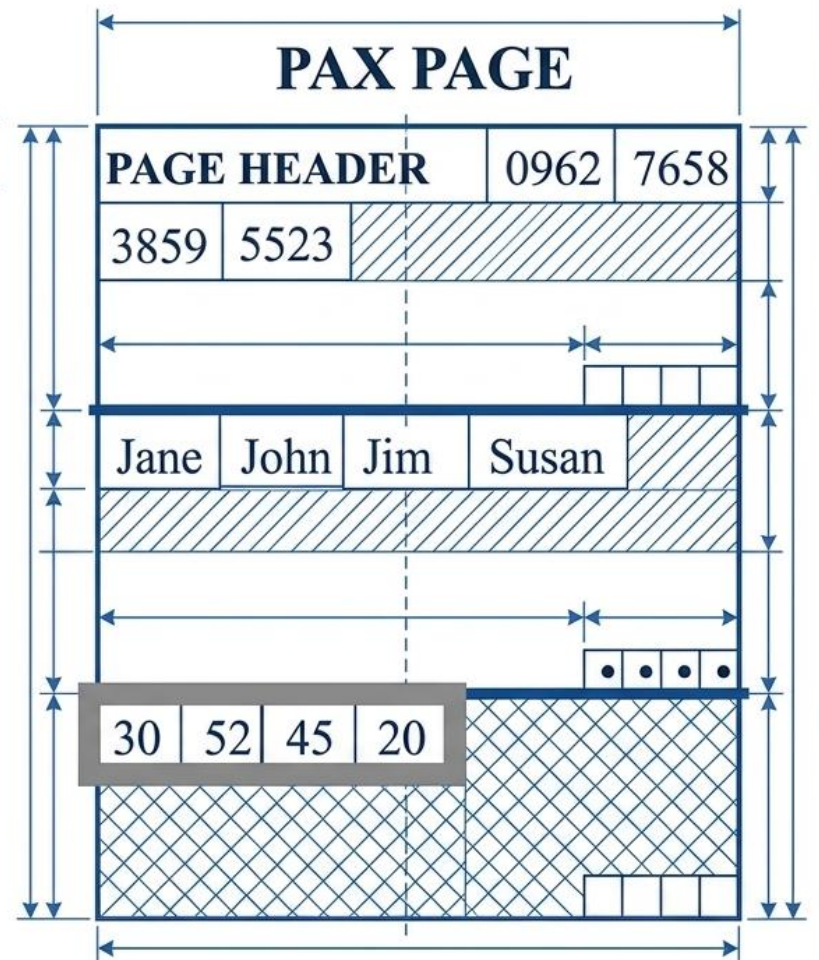
The Problem with NSM

Random L1 cache access and heavy memory stalls occur when searching specific fields across rows within a page.

The PAX Solution (Partition Attributes Across)

Retains the NSM I/O pattern at the disk-to-page level but reorganizes data internally.

Groups data by columns within the page to optimize CPU cache-to-RAM communication, preventing irrelevant data from flooding the L1 cache.



Execution Paradigm I: Vectorized Processing

The Volcano Iterator (Legacy)

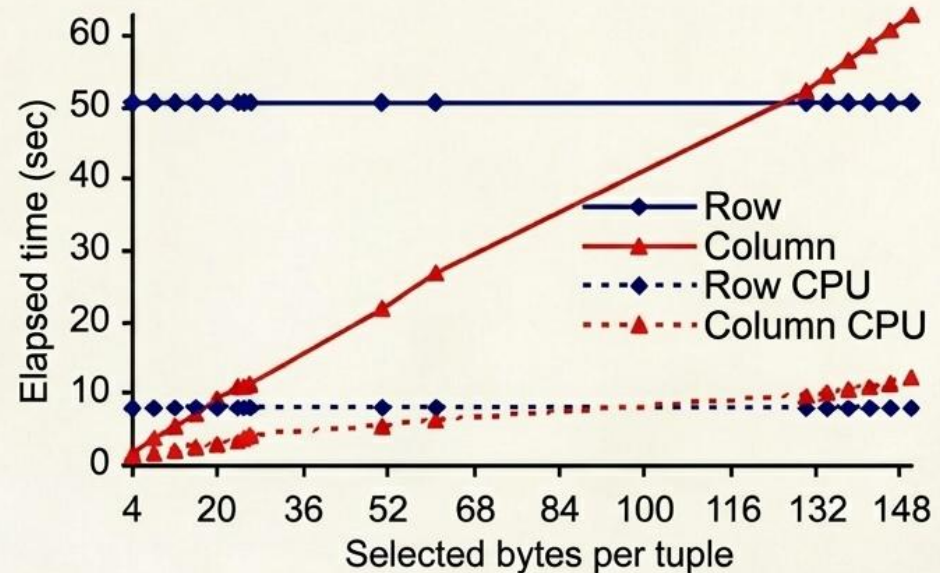
Processes data row-by-row, incurring massive function-call overhead and constant cache misses.

Vectorization (Modern)

Data is loaded into CPU registers in tight, contiguous batches (vectors) of thousands of values.

Hardware Synergy

Exploits SIMD (Single Instruction, Multiple Data). A single instruction simultaneously operates on an entire vector of columnar data, mathematically eliminating instruction dispatch overhead.

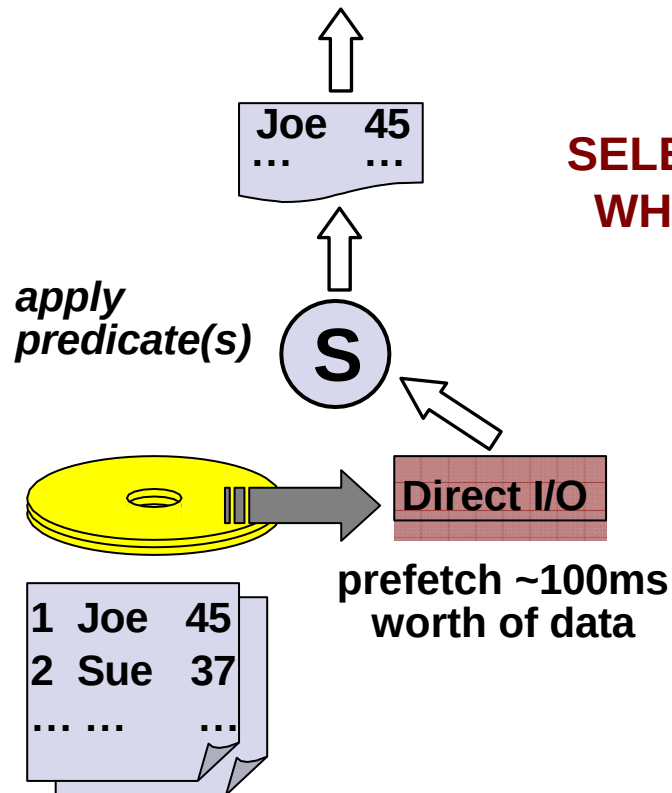


Column-scanner implementation

“Performance Tradeoffs in Read-Optimized Databases”

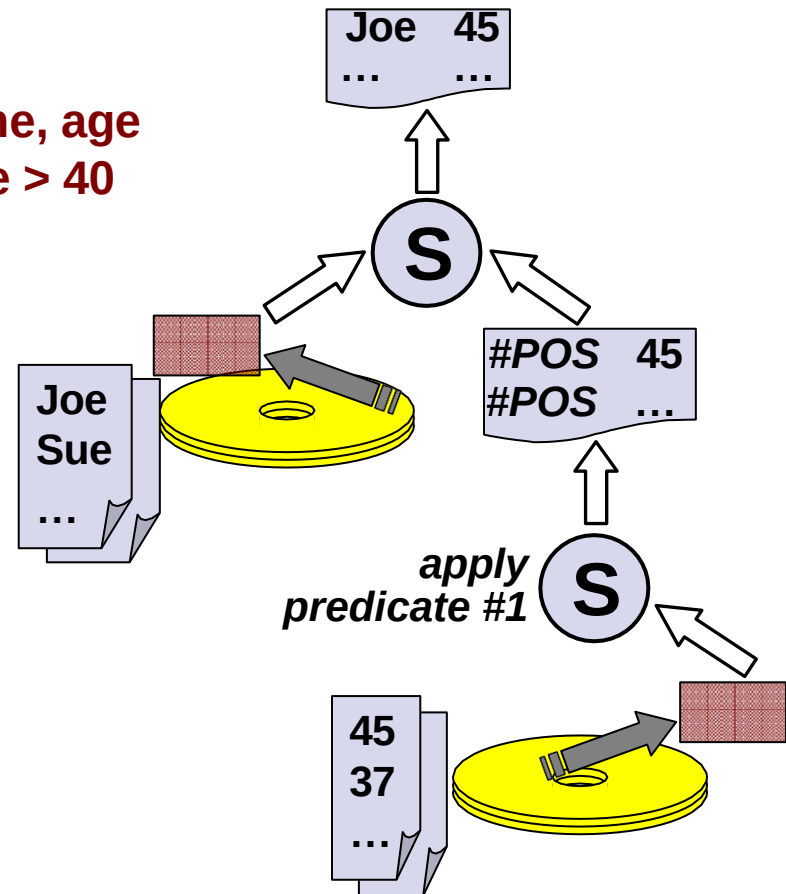
Harizopoulos, Liang, Abadi, Madden, VLDB'06

row scanner



**SELECT name, age
WHERE age > 40**

column scanner



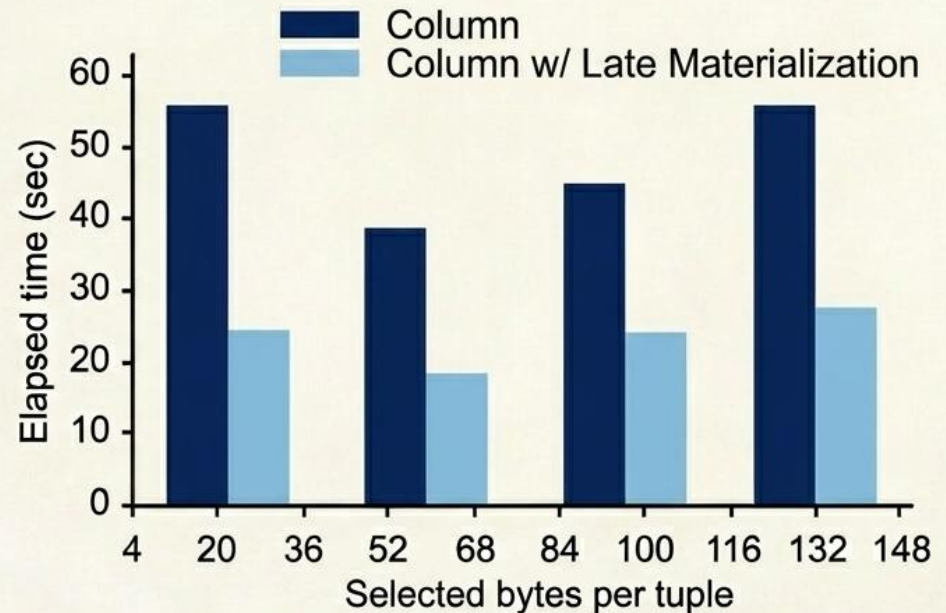
Execution Paradigm II: Late Materialization

✗ Early Materialization (The Flaw)

Extracting fields from isolated files and immediately stitching them back into a logical row before filtering forces massive data movement across the memory bus.

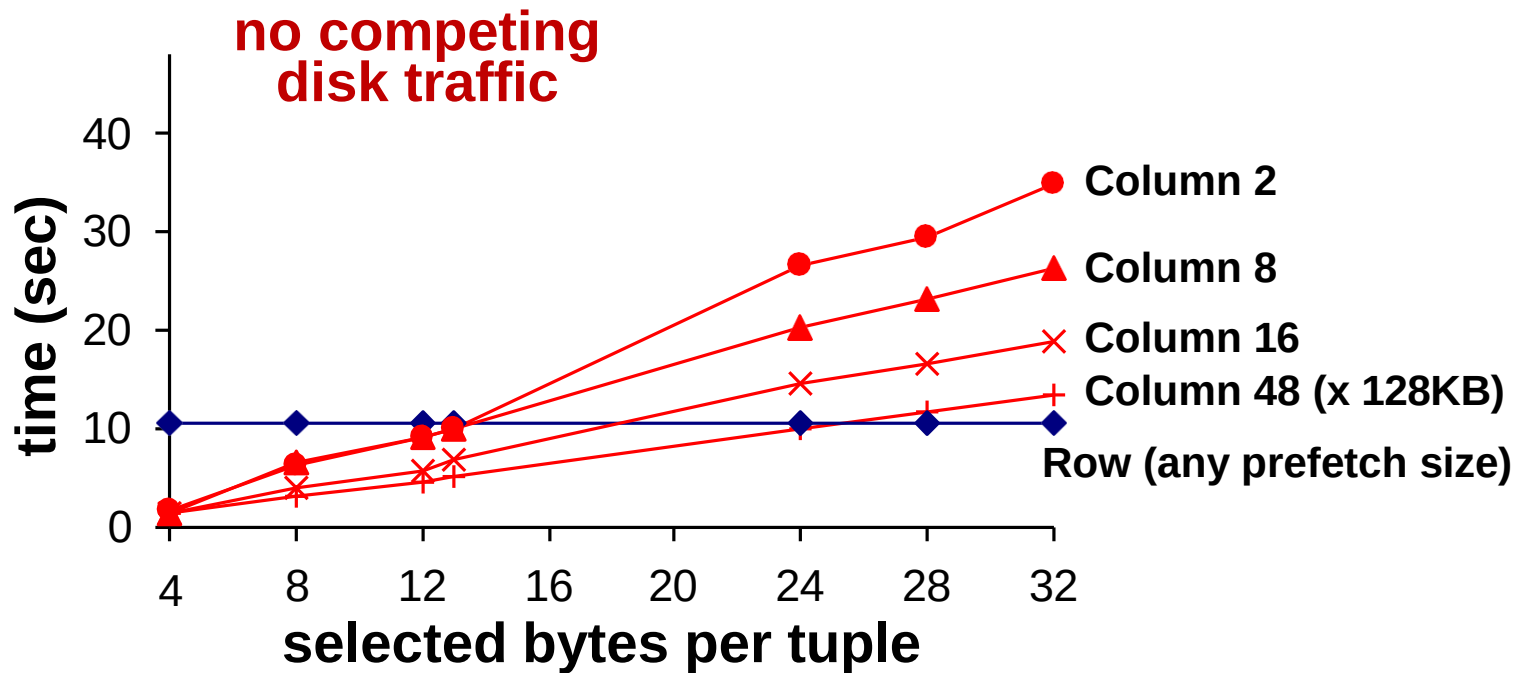
✓ Late Materialization (The Fix)

1. Keep data in its native, compressed columnar format.
2. Apply filters and aggregations directly to the compressed columns using implicit positions (Virtual IDs).
3. Only stitch the remaining valid tuples back together via position matching at the very end of the query plan.



SSBM query execution times...
Column w/ Late Materialization

Varying prefetch size



- 1 No prefetching hurts columns in single scans

Seminal Architecture: C-Store

ROS (Read-Optimized Store)

The core engine. Disk-based, heavily compressed, sorted segments. Strictly immutable.

WOS (Write-Optimized Store)

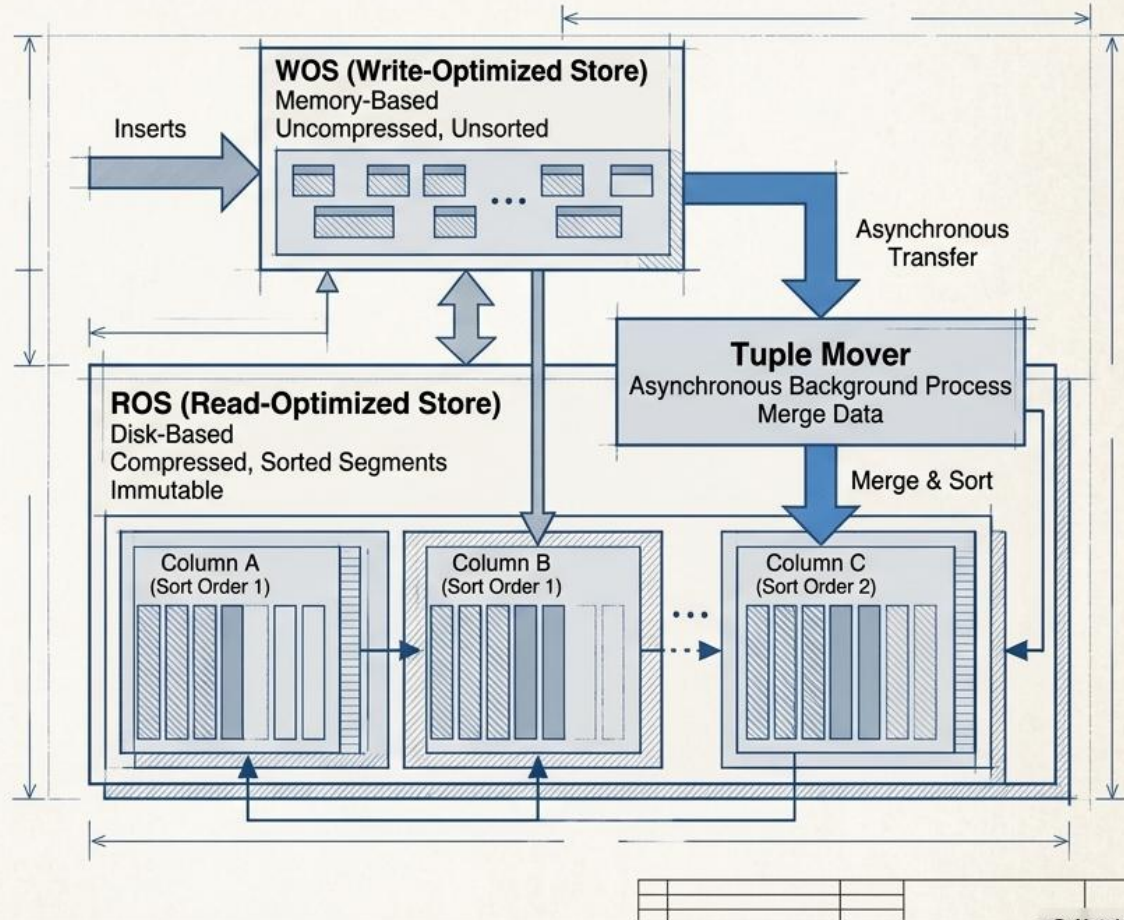
Memory-based, uncompressed, and unsorted. Exists solely to absorb high-velocity inserts without bottlenecking the ROS.

Tuple Mover

The background asynchronous process that merges WOS data into the ROS.

Overlapping Projections

C-Store discarded traditional secondary indexes in favor of redundantly storing columns in multiple different sort orders.



NotebookLM

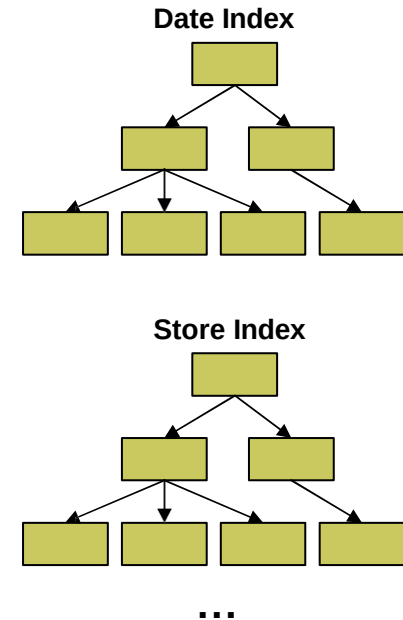
Simulate a Column-Store inside a Row-Store

Date	Store	Product	Customer	Price
01/01	BOS	Table	Mesa	\$20
01/01	NYC	Chair	Lutz	\$13
01/01	BOS	Bed	Mudd	\$79

**Option A:
Vertical Partitioning**

Date		Store		Product		Customer		Price	
TID	Value	TID	Value	TID	Value	TID	Value	TID	Value
1	01/01	1	BOS	1	Table	1	Mesa	1	\$20
2	01/01	2	NY	2	Chair	2	Lutz	2	\$13
3	01/01	3	C	3	Bed	3	Mudd	3	\$79
			BO						
			S						

**Option B:
Index Every Column**



Simulate a Column-Store inside a Row-Store

Date	Store	Product	Customer	Price
01/01	BOS	Table	Mesa	\$20
01/01	NYC	Chair	Lutz	\$13
01/01	BOS	Bed	Mudd	\$79

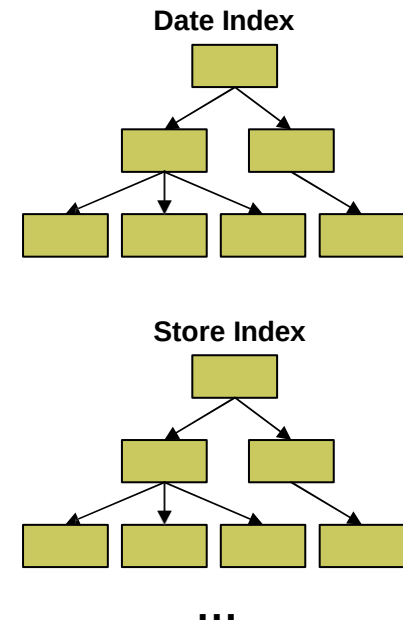
**Option A:
Vertical Partitioning**

Date		
Value	StartPos	Length
01/01	1	3

Can explicitly run-length encode date

Store		Product		Customer		Price	
TID	Value	TID	Value	TID	Value	TID	Value
1	BOS	1	Table	1	Mesa	1	\$20
2	NY	2	Chair	2	Lutz	2	\$13
3	C	3	Bed	3	Mudd	3	\$79
	BO						
	S						

**Option B:
Index Every Column**



What about vertical partitioning? (it does not work with ad-hoc queries)

The Vertical Partitioning Fallacy



The Hypothesis

If column stores are just vertical slices, vertically partitioning a traditional row-store (like PostgreSQL) should yield the exact same OLAP performance.



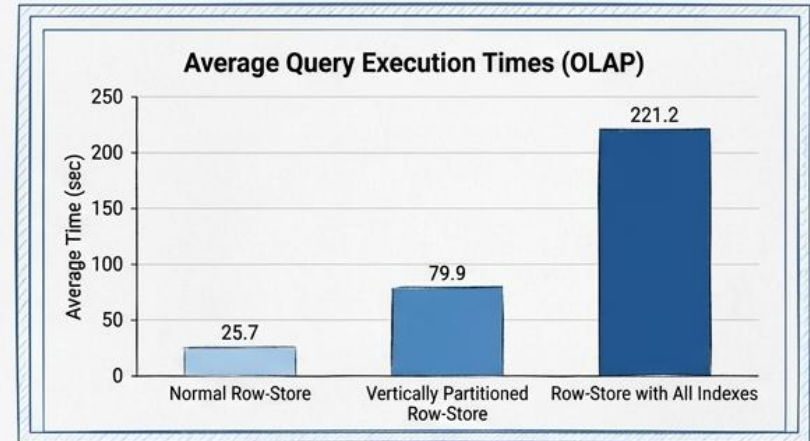
The Reality

Benchmarks prove vertically partitioned row-stores actually perform worse than standard row-stores for analytical queries.

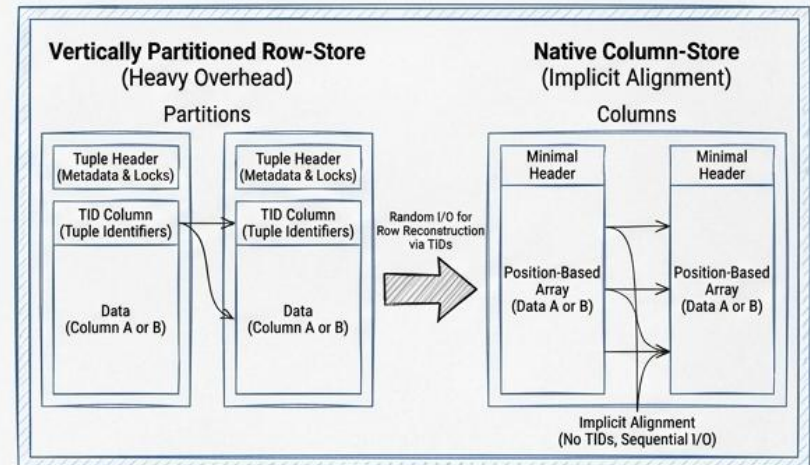


The Mechanics of the Failure

Row-stores mandate heavy Tuple IDs and header metadata for every single partition. Reconstructing the logical row requires joining these partitions via TIDs, triggering catastrophic random I/O. Native column-stores avoid this entirely via implicit, position-based arrays.



Benchmarks prove vertical partitioning worsens OLAP performance.



What's Going On? Vertical Partitions

- 1 Vertical partitions in row-stores:
 - 1 Work well when workload is known
 - 1 ..and queries access disjoint sets of columns
 - 1 See automated physical design
- 1 Do not work well as full-columns
 - 1 TupleID overhead significant
 - 1 Excessive joins

Tuple Header	TID	Column Data
	1	
	2	
	3	

“Column-Stores vs. Row-Stores:
How Different Are They Really?”
Abadi, Madden, and Hachem.
SIGMOD 2008.

Queries touch 3-4 foreign keys in fact table,
1-2 numeric columns

Complete fact table takes up ~4 GB
(compressed)

Vertically partitioned tables take up 0.7-1.1
GB (compressed)

What's Going On? All Indexes Case

1 Tuple construction

1 Common type of query:

```
SELECT store_name, SUM(revenue)
FROM Facts, Stores
WHERE fact.store_id = stores.store_id
      AND stores.country = "Canada"
GROUP BY store_name
```

1 Result of lower part of query plan is a set of TIDs that passed all predicates

1 Need to extract SELECT attributes at these TIDs

1 BUT: index maps value to TID

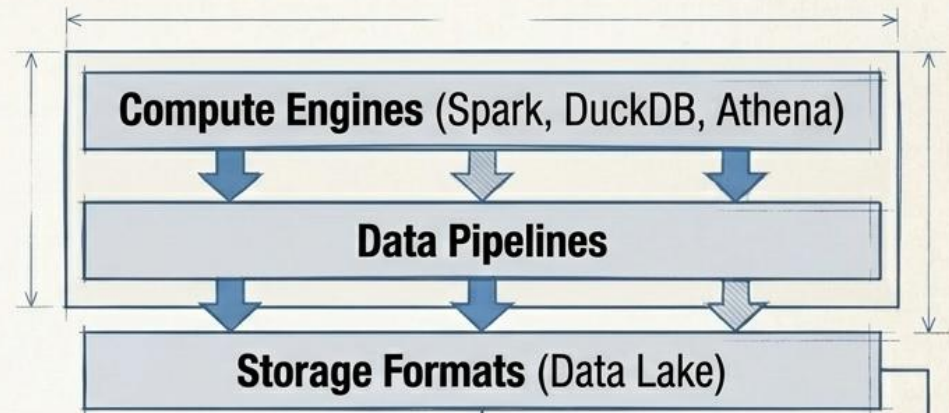
1 You really want to map TID to value (i.e., a vertical partition)

Tuple construction is SLOW



Storage Decoupling: Open Columnar Formats

Formats define serialization;
DBMS engines provide execution,
indexing, and state.



Apache Parquet

The dominant standard for Data Lakes. Disk-based, highly compressed, deeply integrated with cloud query engines.

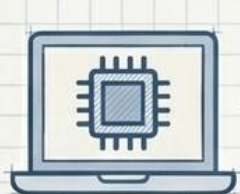
Apache ORC

Hadoop ecosystem native, structurally optimized for Hive and legacy big data pipelines.

Apache Arrow

An in-memory columnar format. Designed for zero-copy data interchange between analytical systems (Pandas, ML pipelines) to eliminate serialization overhead entirely.

Architectural Scale: Embedded to Distributed



Embedded



Hybrid



Distributed

Embedded (DuckDB) 🦆

In-process, zero-server footprint. Vectorized execution optimized for out-of-core processing on single nodes. Excellent for local ETL and interactive data science notebooks.

Hybrid (MotherDuck) 🦆

Local-first computation that seamlessly spills over to centralized cloud datasets, bridging the gap between local speed and shared state.

Distributed (ClickHouse) 📊

Horizontally scalable, shared-nothing architecture. Engineered for high-concurrency writes and sub-second aggregations across trillions of rows.



Diagnostic Synthesis: Row vs. Column

	Row-Oriented Layout (OLTP)	Column-Oriented Layout (OLAP)
Storage Layout	Contiguous records	Contiguous fields
I/O Profile	Scans entire table	Selectively prunes files
Compression	Low (Mixed-types)	High (5-10x, homogeneous)
Optimal Queries	Point lookups (SELECT * WHERE id=X)	Bulk Aggregations (SUM, GROUP BY)
Write Profile	Single-row ACID inserts	Large batched, immutable appends



The Path Forward: System Convergence

No Silver Bullet

Columnar architectures are the undisputed kings of analytics, but structurally fail at transactional integrity, SELECT *, and high-frequency single-row mutations.

OLTP Row-Store
(e.g., PostgreSQL)

**CDC (Change Data Capture)
Pipeline**

Seamless Data Flow
for Real-Time Analytics

OLAP Columnar Engine
(e.g., ClickHouse)

The HTAP Reality

Hybrid Transactional/Analytical Processing is the new standard. Organizations no longer choose one or the other.

Unified Data Stacks

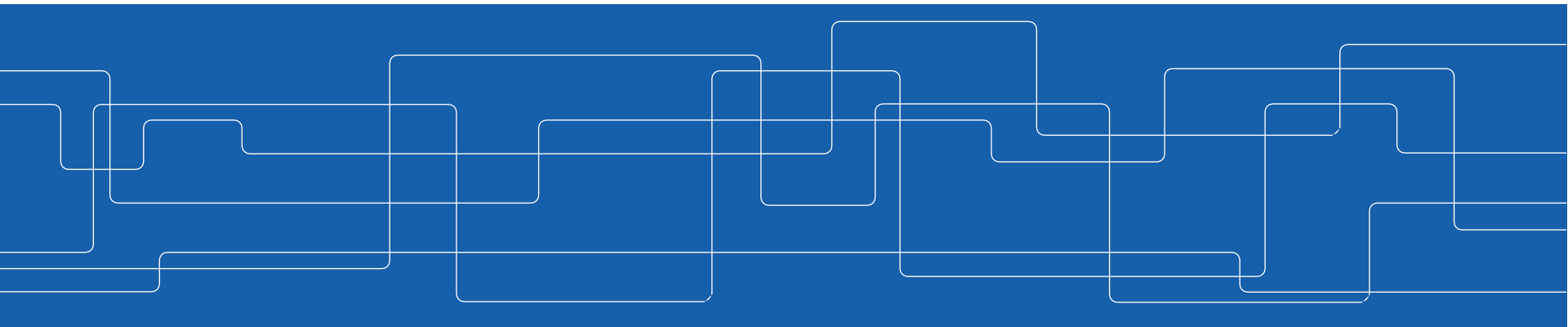
Modern systems utilize CDC pipelines or built-in replication to seamlessly capture OLTP row-data and stream it into OLAP columnar platforms for real-time analytics.

List of column-store systems

- 1 Cantor (history)
- 1 Sybase IQ
- 1 SenSage (former Addamark Technologies)
- 1 Kdb
- 1 1010data
- 1 MonetDB
- 1 C-Store/Vertica
- 1 X100/VectorWise
- 1 KickFire
- 1 SAP Business Accelerator
- 1 Infobright
- 1 ParAccel
- 1 Exasol



Data Stream Processing



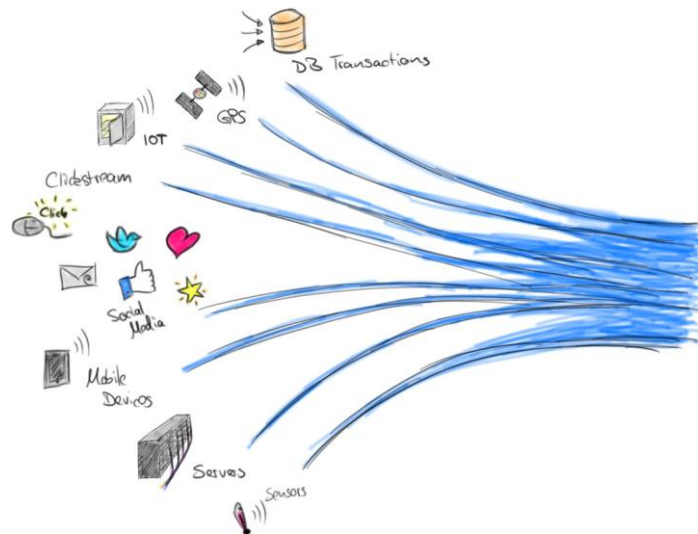
Stream Processing (1/4)

- **Stream processing** is the act of **continuously** incorporating **new data** to compute a result.



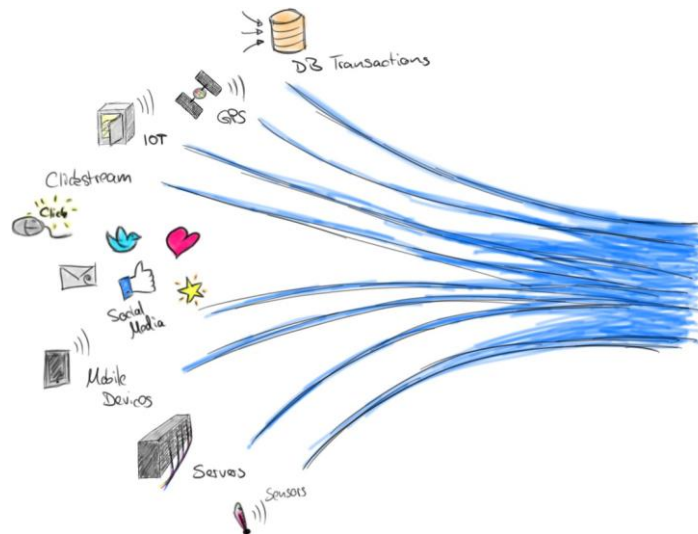
Stream Processing (2/4)

- ▶ The input data is **unbounded**.
 - A **series of events**, no predetermined **beginning or end**.



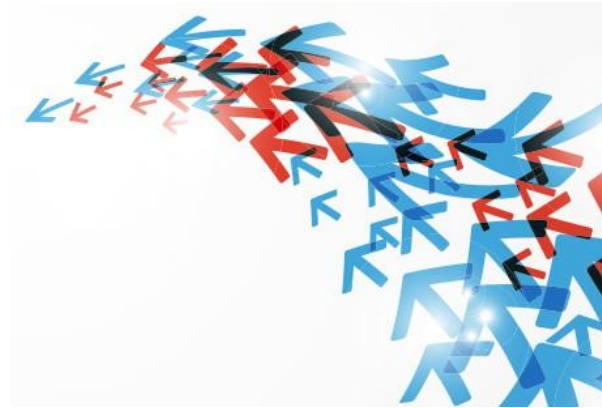
Stream Processing (2/4)

- ▶ The **input data** is **unbounded**.
 - A **series of events**, no predetermined **beginning or end**.
 - E.g., credit card transactions, clicks on a website, or sensor readings from IoT devices.



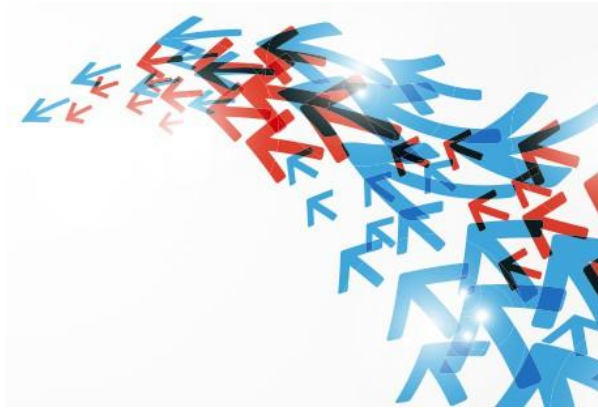
Stream Processing (3/4)

- ▶ **User applications** can then compute **various queries** over this stream of events.



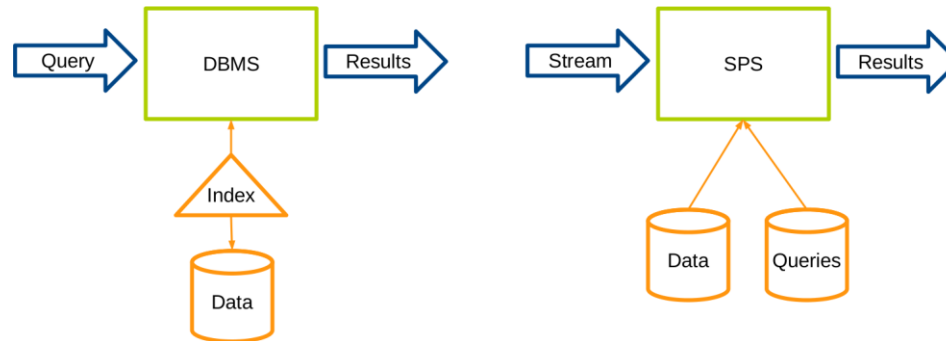
Stream Processing (3/4)

- ▶ **User applications** can then compute **various queries** over this stream of events.
 - E.g., **tracking** a running count of each type of event, or **aggregating** them into hourly windows.



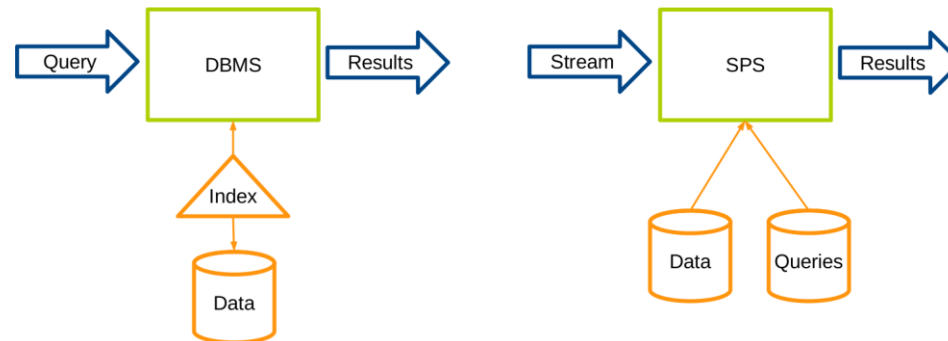
Stream Processing (4/4)

- ▶ Database Management Systems (DBMS): **data-at-rest** analytics
 - **Store** and **index** data before processing it.
 - Process data only when **explicitly** asked by the users.

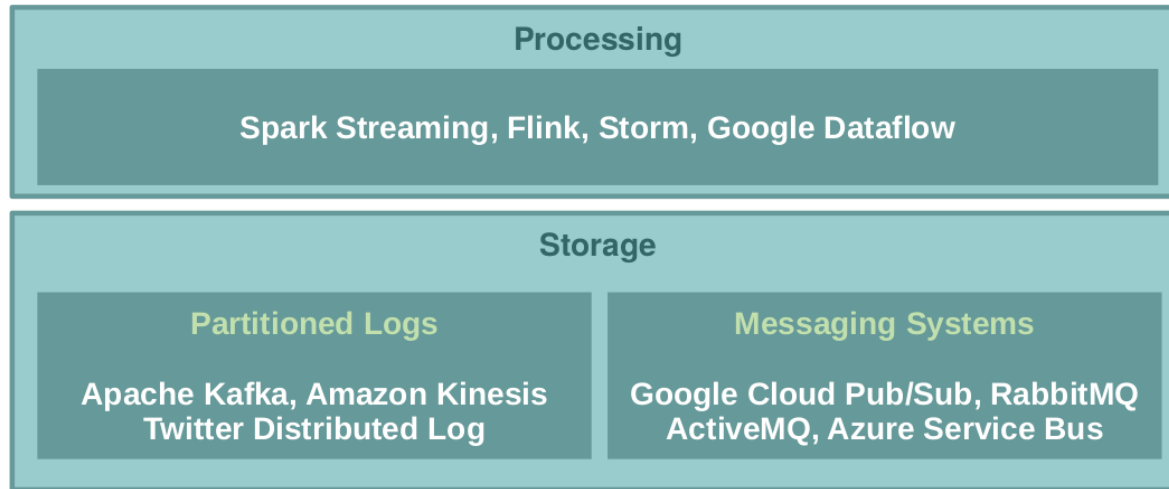


Stream Processing (4/4)

- ▶ Database Management Systems (DBMS): **data-at-rest** analytics
 - **Store** and **index** data before processing it.
 - Process data only when **explicitly** asked by the users.
- ▶ Stream Processing Systems (SPS): **data-in-motion** analytics
 - Processing information as it **flows**, **without storing** them persistently.



Stream Processing Systems Stack



Data Stream Storage

The Problem

- ▶ We need disseminate streams of events from various producers to various consumers.

Data Producers



push



pull

Data Consumers



ETL



complex
event
processing



User Transactions



Example

- ▶ Suppose you have a [website](#), and every time someone [loads a page](#), you send a [viewed page](#) event to consumers.



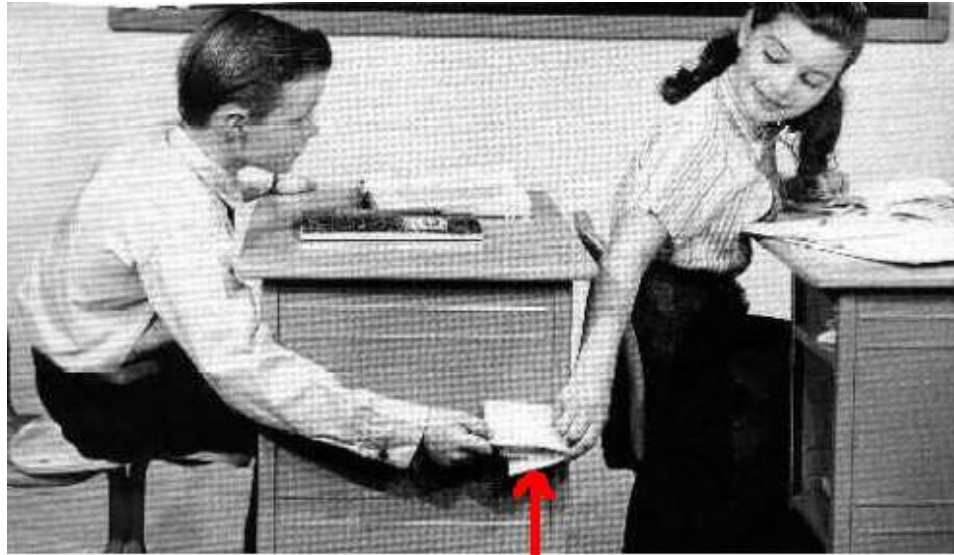
Example

- ▶ Suppose you have a **website**, and every time someone **loads a page**, you send a **viewed page** event to consumers.
- ▶ The consumers may do any of the following:
 - **Store** the message in HDFS for future analysis
 - **Count page** views and update a dashboard
 - Trigger an **alert** if a page view fails
 - Send an **email** notification to another user



Possible Solution?

- ▶ Messaging systems



Message

www.defit.org

What is Messaging System?

- ▶ **Messaging system** is an approach to **notify consumers** about new events.



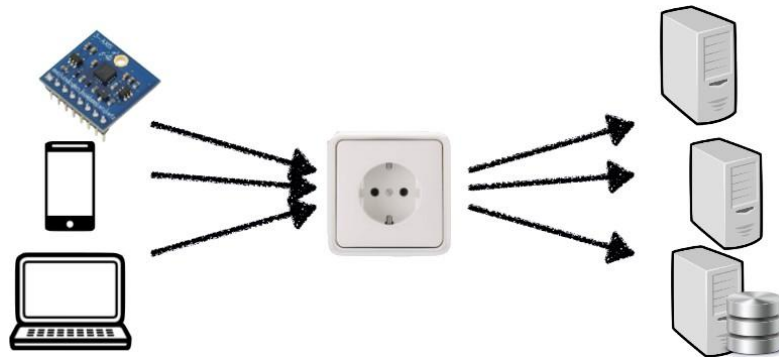
What is Messaging System?

- ▶ **Messaging system** is an approach to **notify consumers** about new events.
- ▶ **Messaging systems**
 - **Direct** messaging
 - Message **brokers**



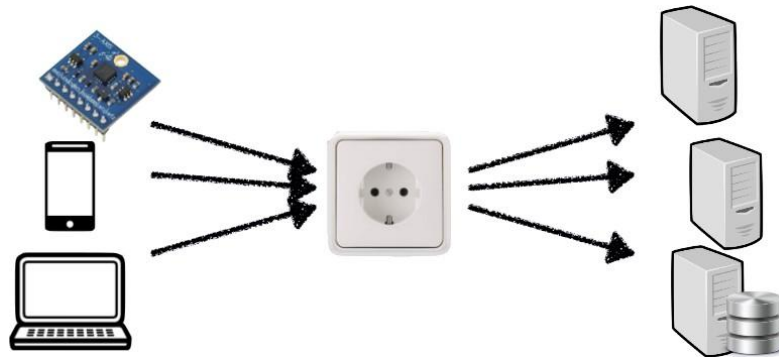
Direct Messaging (1/2)

- ▶ Necessary in **latency critical** applications (e.g., remote surgery).
- ▶ A **producer** sends a message containing the event, which is **pushed** to consumers.



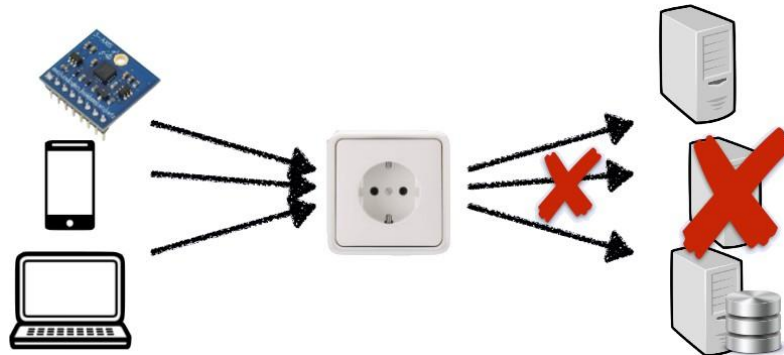
Direct Messaging (1/2)

- ▶ Necessary in **latency critical** applications (e.g., remote surgery).
- ▶ A **producer** sends a message containing the event, which is **pushed** to consumers.
- ▶ Both **consumers and producers** have to be **online at the same time**.



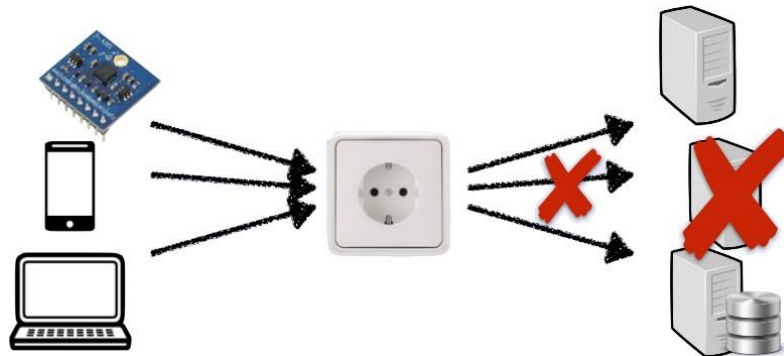
Direct Messaging (2/2)

- What happens if a consumer crashes or temporarily goes offline? (not durable)



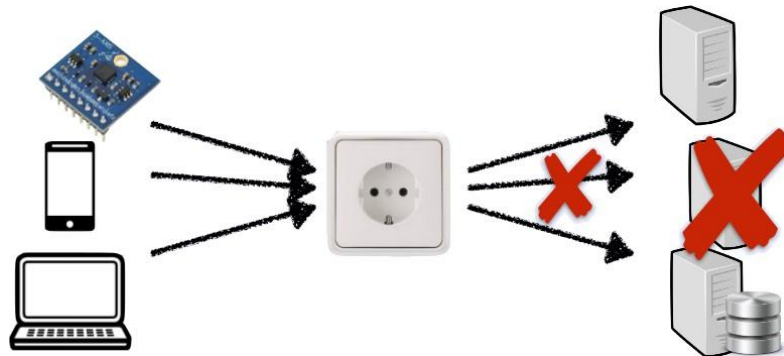
Direct Messaging (2/2)

- ▶ What happens if a consumer crashes or temporarily goes offline? (not durable)
- ▶ What happens if producers send messages faster than the consumers can process?



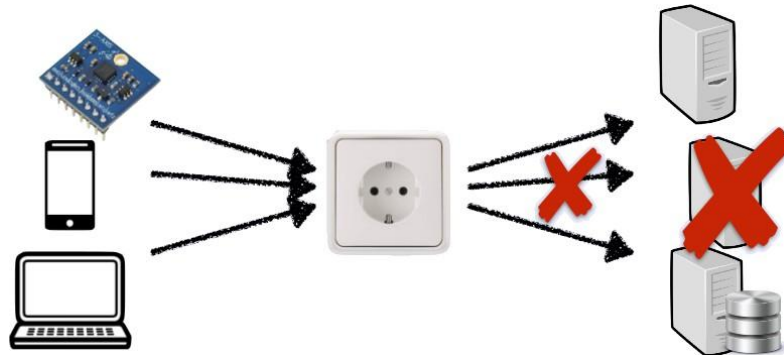
Direct Messaging (2/2)

- ▶ What happens if a consumer crashes or temporarily goes offline? (not durable)
- ▶ What happens if producers send messages faster than the consumers can process?
 - Dropping messages
 - Backpressure

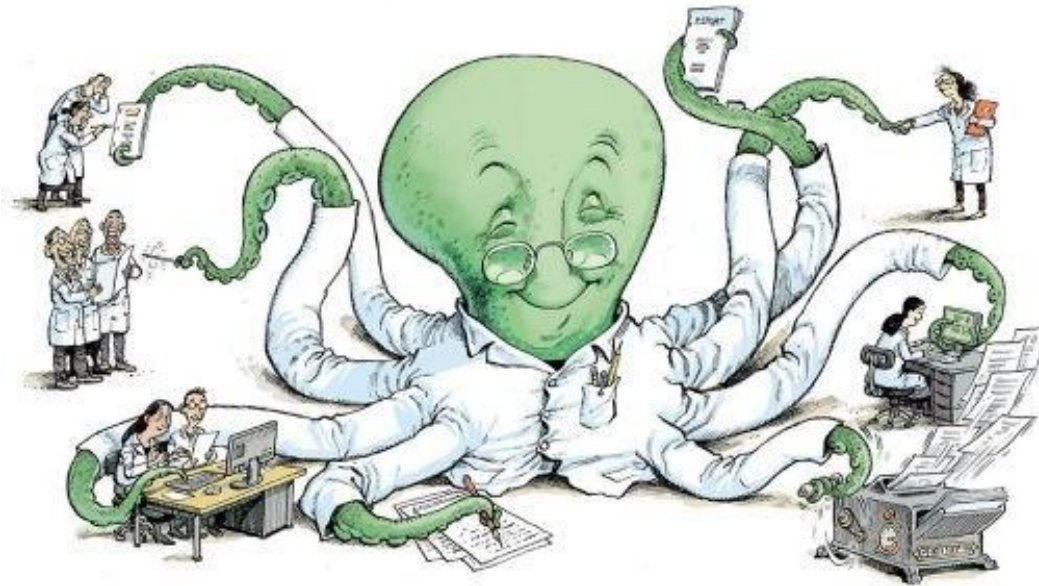


Direct Messaging (2/2)

- ▶ What happens if a consumer crashes or temporarily goes offline? (not durable)
- ▶ What happens if producers send messages faster than the consumers can process?
 - Dropping messages
 - Backpressure
- ▶ We need message brokers that can log events to process at a later time.



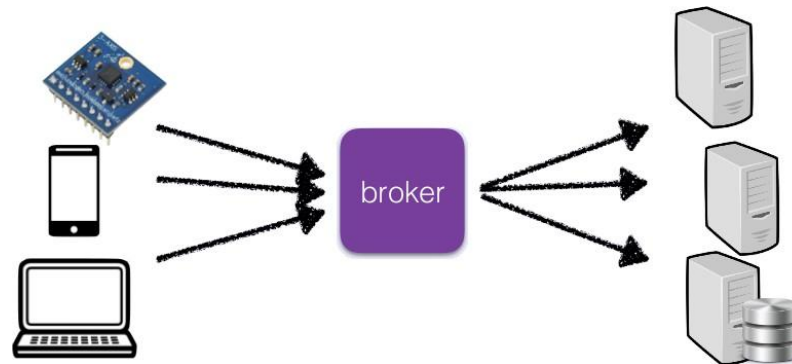
Message Broker



[<https://bluesyemre.com/2018/10/16/thousands-of-scientists-publish-a-paper-every-five-days>]

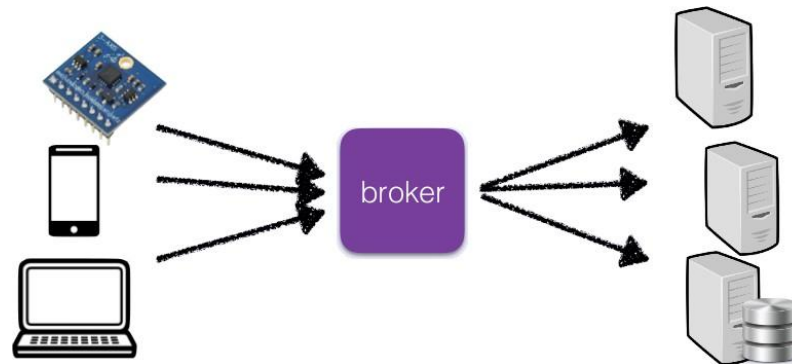
Message Broker

- ▶ A message broker decouples the producer-consumer interaction.
- ▶ It runs as a server, with producers and consumers connecting to it as clients.



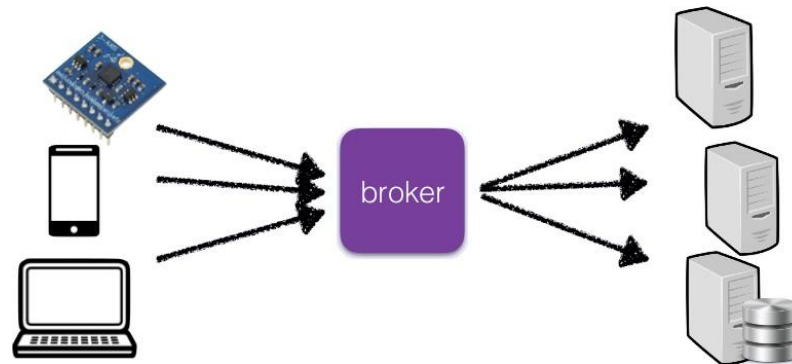
Message Broker

- A **message broker** decouples the **producer-consumer** interaction.
- It runs as a **server**, with **producers** and **consumers** connecting to it as **clients**.
- **Producers** write messages to the broker, and **consumers** receive them by reading them from the broker.



Message Broker

- ▶ A **message broker** decouples the **producer-consumer** interaction.
- ▶ It runs as a **server**, with **producers** and **consumers** connecting to it as **clients**.
- ▶ **Producers** write messages to the broker, and **consumers** receive them by reading them from the broker.
- ▶ **Consumers** are generally **asynchronous**.



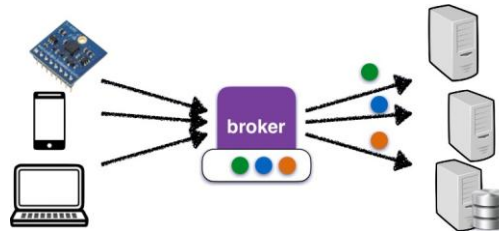
Message Broker (2/2)

- ▶ When **multiple consumers** read messages in the **same topic**.



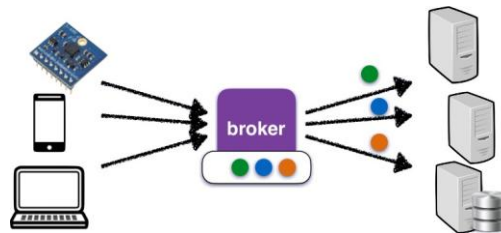
Message Broker (2/2)

- ▶ When **multiple consumers** read messages in the **same topic**.
- ▶ **Load balancing**: each message is delivered to **one** of the consumers.

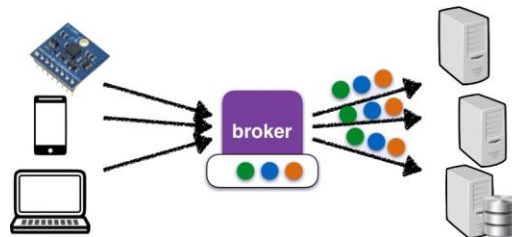


Message Broker (2/2)

- ▶ When **multiple consumers** read messages in the **same topic**.
- ▶ **Load balancing**: each message is delivered to **one** of the consumers.



- ▶ **Fan-out**: each message is delivered to **all** of the consumers.



Partitioned Logs (1/2)

- ▶ In typical message brokers, once a message is **consumed**, it is **deleted**.



Partitioned Logs (1/2)

- ▶ In typical message brokers, once a message is **consumed**, it is **deleted**.
- ▶ Log-based message brokers **durably** store all events in a sequential **log**.



Partitioned Logs (1/2)

- ▶ In typical message brokers, once a message is **consumed**, it is **deleted**.
- ▶ **Log-based message brokers** **durably** store all events in a sequential **log**.
- ▶ A **log** is an **append-only** sequence of records on **disk**.



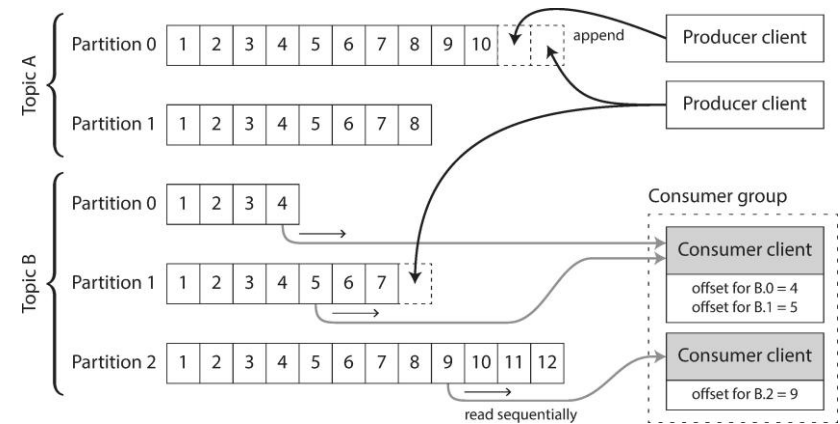
Partitioned Logs (1/2)

- ▶ In typical message brokers, once a message is **consumed**, it is **deleted**.
- ▶ **Log-based message brokers** **durably** store all events in a sequential **log**.
- ▶ A **log** is an **append-only** sequence of records on **disk**.
- ▶ A **producer** sends a message by **appending** it to the end of the log.
- ▶ A **consumer** receives messages by reading the log **sequentially**.



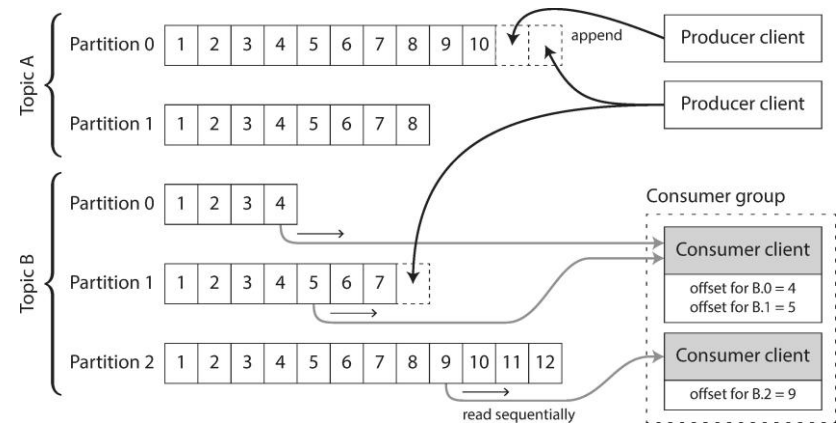
Partitioned Logs (2/2)

- To **scale up** the system, logs can be **partitioned** hosted on **different machines**.



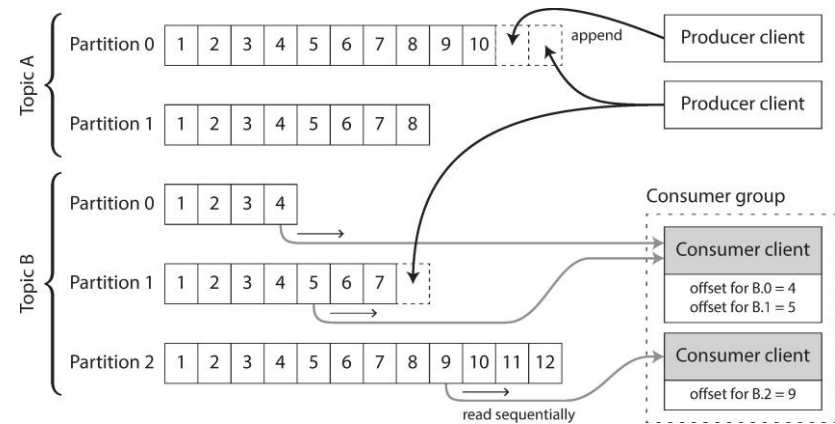
Partitioned Logs (2/2)

- ▶ To **scale up** the system, logs can be **partitioned** hosted on **different machines**.
- ▶ Each **partition** can be read and written **independently** of others.



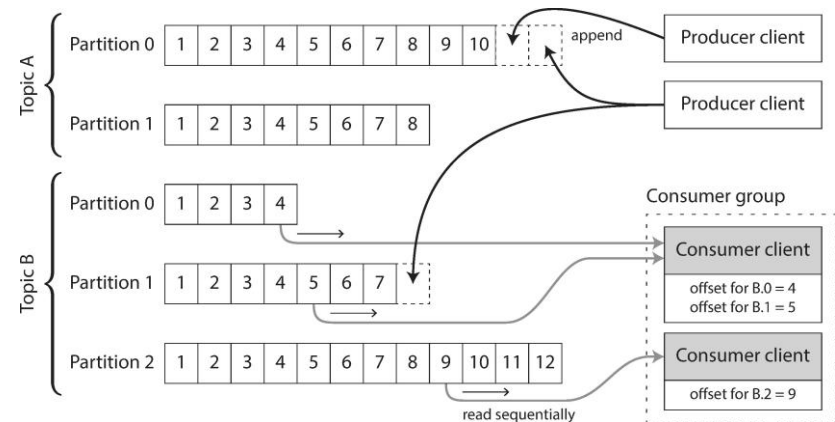
Partitioned Logs (2/2)

- To **scale up** the system, logs can be **partitioned** hosted on **different machines**.
- Each **partition** can be read and written **independently** of others.
- A **topic** is a **group of partitions** that all carry messages of the **same type**.



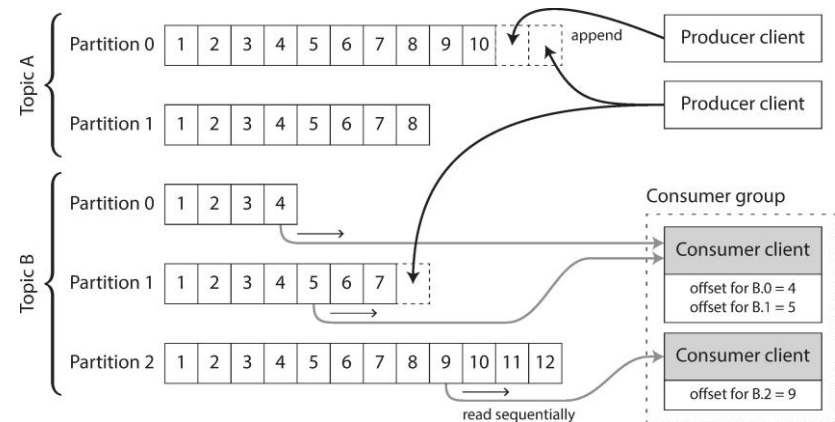
Partitioned Logs (2/2)

- To **scale up** the system, logs can be **partitioned** hosted on **different machines**.
- Each **partition** can be read and written **independently** of others.
- A **topic** is a **group of partitions** that all carry messages of the **same type**.
- Within each partition, the broker assigns a **monotonically increasing sequence number (offset)** to every message



Partitioned Logs (2/2)

- To **scale up** the system, logs can be **partitioned** hosted on **different machines**.
- Each **partition** can be read and written **independently** of others.
- A **topic** is a **group of partitions** that all carry messages of the **same type**.
- Within each partition, the broker assigns a **monotonically increasing sequence number (offset)** to every message
- **No ordering** guarantee **across partitions**.

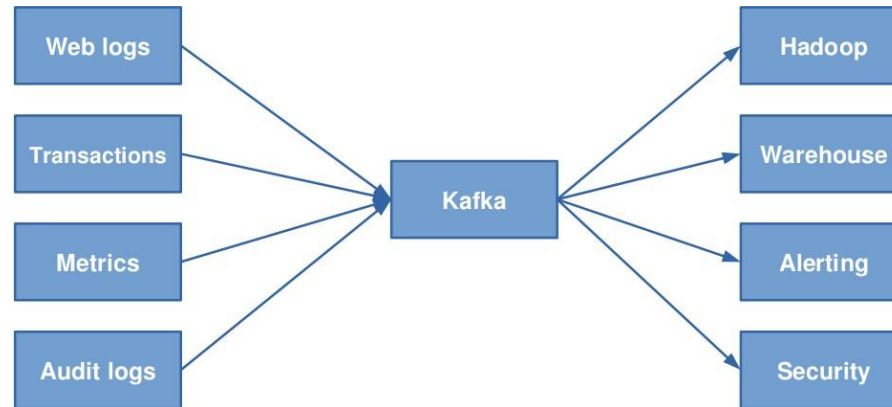


Kafka - A Log-Based Message Broker



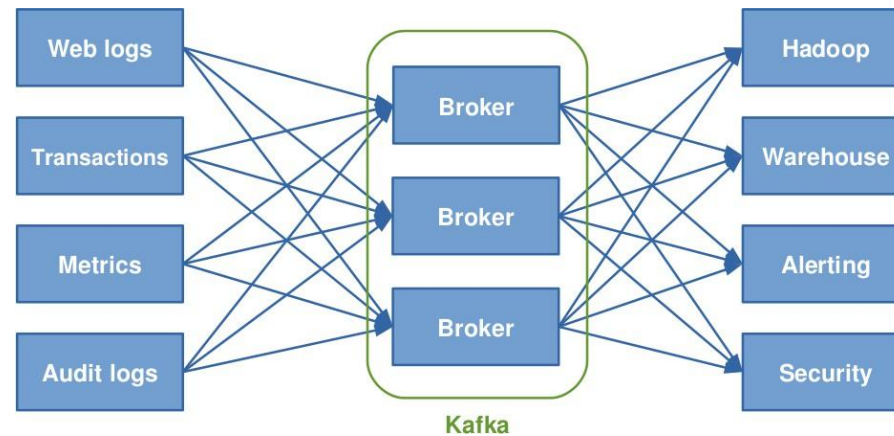
Kafka (1/5)

- **Kafka** is a distributed, topic oriented, partitioned, replicated commit **log service**.



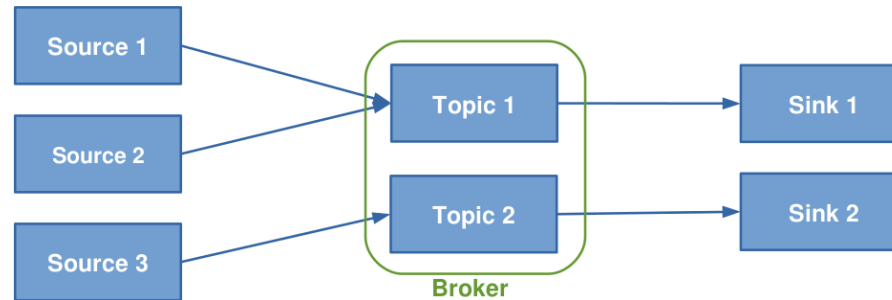
Kafka (2/5)

- Kafka is a **distributed**, topic oriented, partitioned, replicated commit **log** service.



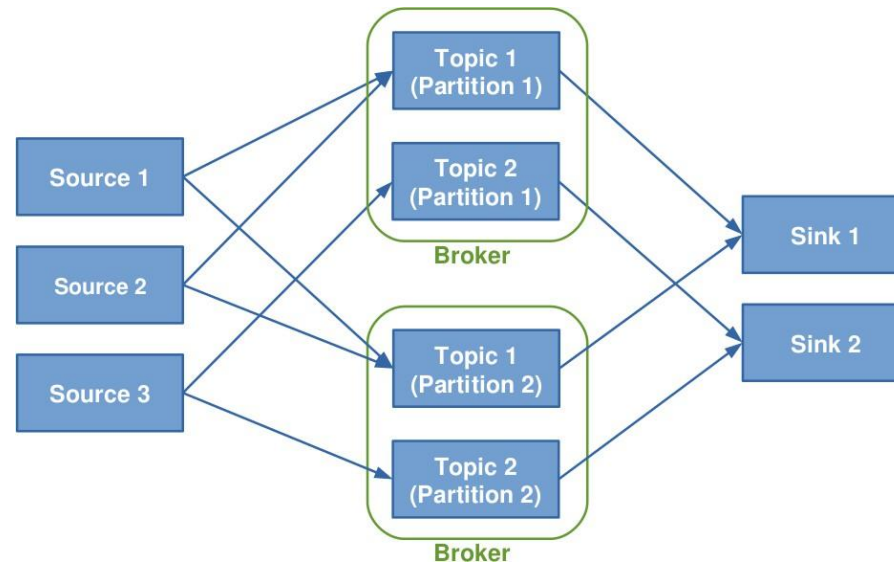
Kafka (3/5)

- Kafka is a distributed, topic oriented, partitioned, replicated commit log service.



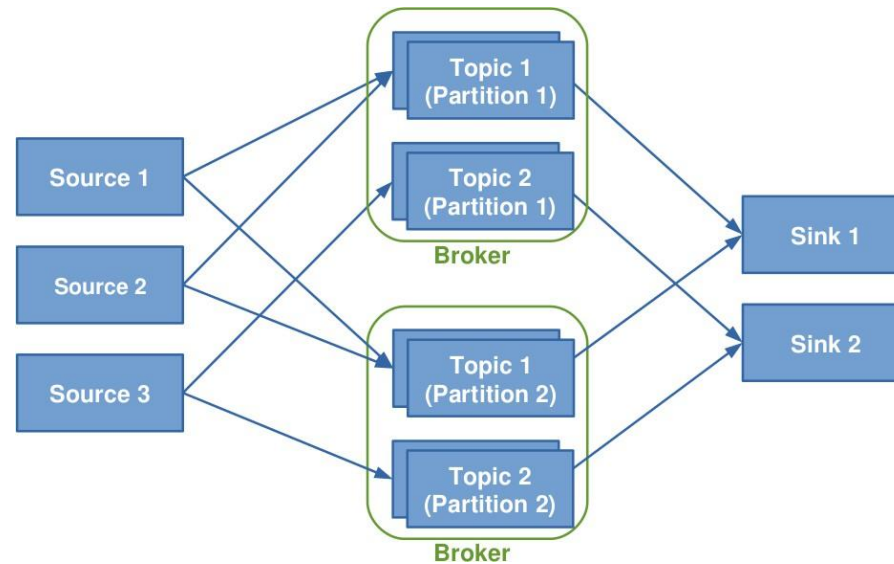
Kafka (4/5)

- Kafka is a distributed, topic oriented, partitioned, replicated commit log service.



Kafka (5/5)

- Kafka is a distributed, topic oriented, partitioned, replicated commit log service.



Logs, Topics and Partition (1/5)

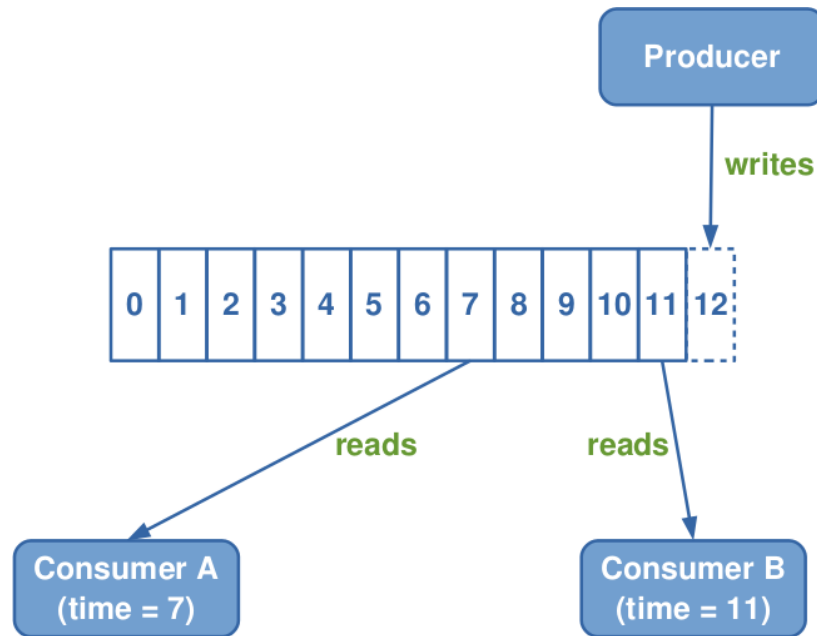
- Kafka is about **logs**.
- **Topics** are **queues**: a **stream of messages** of a **particular type**

```
jkreps-mn:~ jkreps$ tail -f -n 20 /var/log/apache2/access_log
::1 - - [23/Mar/2014:15:07:00 -0700] "GET /images/apache_feather.gif HTTP/1.1" 200 4128
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/producer_consumer.png HTTP/1.1" 200 86
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/log_anatomy.png HTTP/1.1" 200 19579
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/consumer-groups.png HTTP/1.1" 200 2682
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/log_compaction.png HTTP/1.1" 200 41414
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /documentation.html HTTP/1.1" 200 189893
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/log_cleaner_anatomy.png HTTP/1.1" 200
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/kafka_log.png HTTP/1.1" 200 134321
::1 - - [23/Mar/2014:15:07:04 -0700] "GET /images/mirror-maker.png HTTP/1.1" 200 17054
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /documentation.html HTTP/1.1" 200 189937
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /styles.css HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/kafka_logo.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/producer_consumer.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/log_anatomy.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/consumer-groups.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/log_cleaner_anatomy.png HTTP/1.1" 304
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/log_compaction.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/kafka_log.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:08:07 -0700] "GET /images/mirror-maker.png HTTP/1.1" 304 -
::1 - - [23/Mar/2014:15:09:55 -0700] "GET /documentation.html HTTP/1.1" 200 195264
```



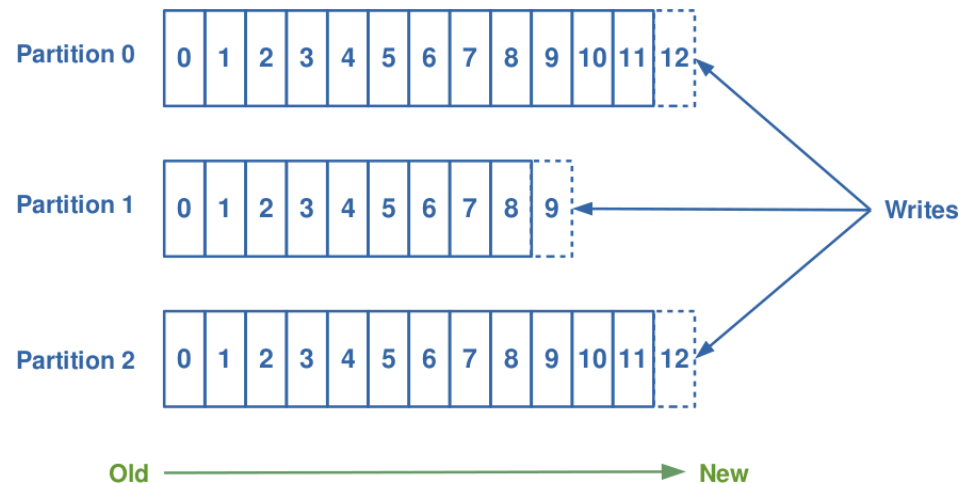
Logs, Topics and Partition (2/5)

- Each **message** is assigned a **sequential id** called an **offset**.



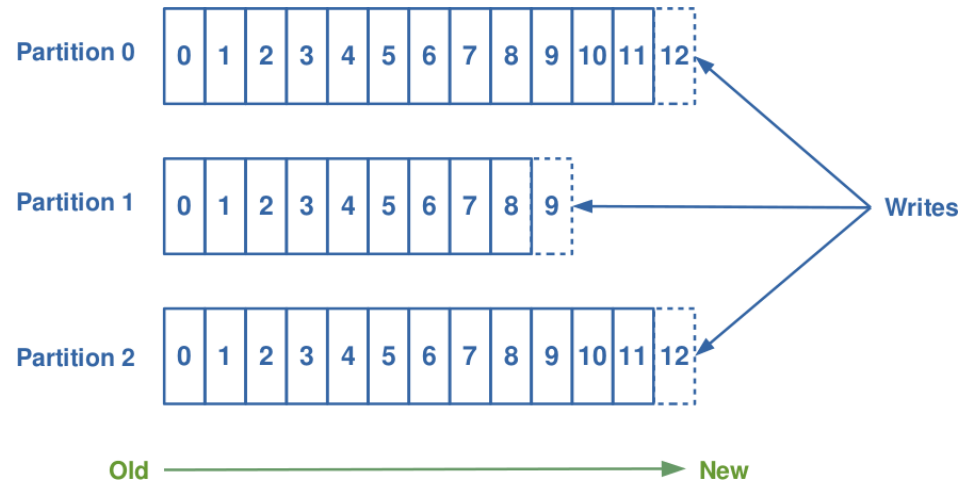
Logs, Topics and Partition (3/5)

- Topics are **logical** collections of **partitions** (the **physical files**).
 - Ordered
 - Append only
 - Immutable



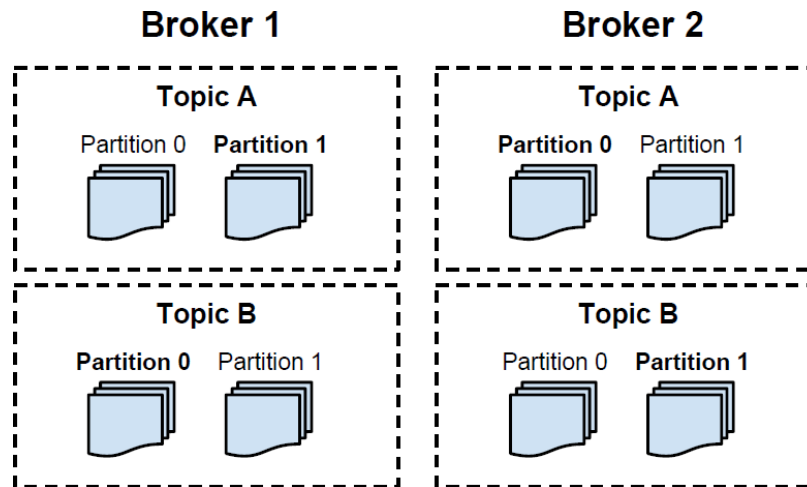
Logs, Topics and Partition (4/5)

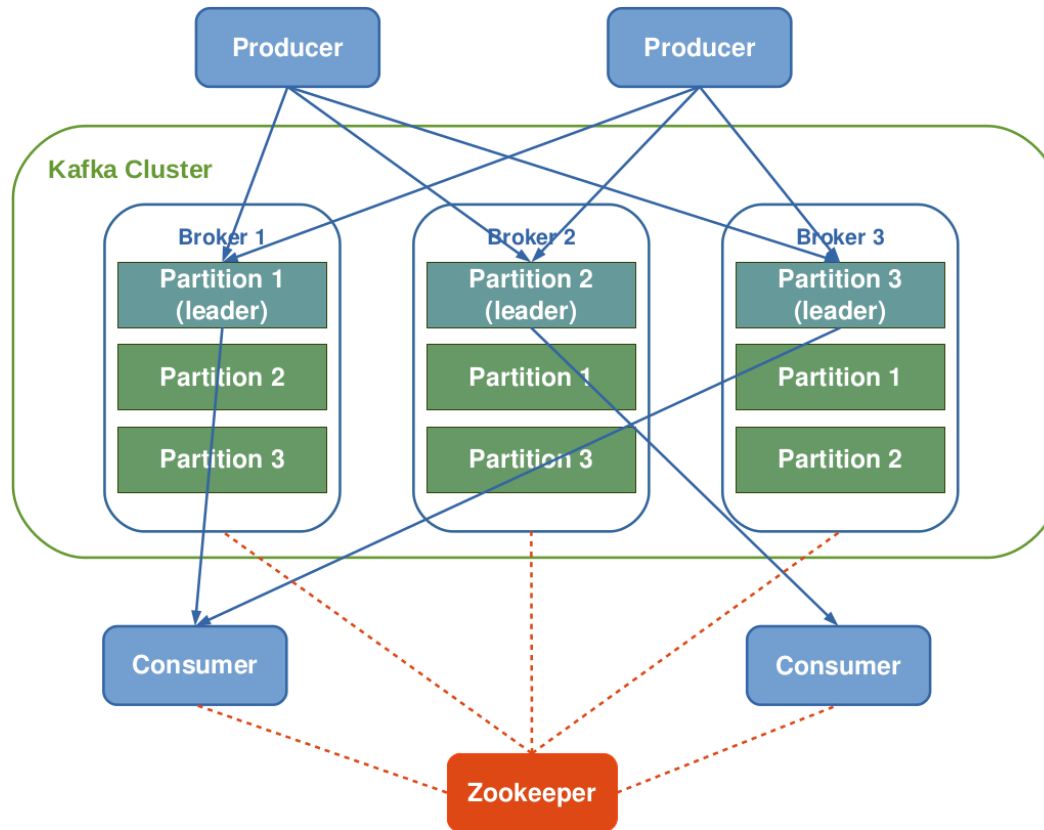
- ▶ Ordering is only **guaranteed within** a **partition** for a **topic**.
- ▶ Messages sent by a **producer** to a particular topic partition will be **appended** in the order they are sent.
- ▶ A **consumer** instance sees messages in the order they are stored in the log.



Logs, Topics and Partition (5/5)

- ▶ Partitions of a topic are replicated: fault-tolerance
- ▶ A broker contains some of the partitions for a topic.
- ▶ One broker is the leader of a partition: all writes and reads must go to the leader.





Coordination

- Kafka uses **Zookeeper** for the following tasks:



Coordination

- ▶ Kafka uses **Zookeeper** for the following tasks:
- ▶ Detecting the **addition** and the **removal** of **brokers** and **consumers**.
- ▶ Keeping track of the **consumed** offset of each partition.



State in Kafka

- ▶ Brokers are **stateless**: **no metadata** for consumers-producers in **brokers**.



State in Kafka

- ▶ Brokers are **sateless**: **no metadata** for consumers-producers in **brokers**.
- ▶ **Consumers** are responsible for keeping track of **offsets**.



State in Kafka

- ▶ Brokers are **stateless**: **no metadata** for consumers-producers in **brokers**.
- ▶ **Consumers** are responsible for keeping track of **offsets**.
- ▶ Messages in queues **expire** based on pre-configured time periods (e.g., once a day).



Delivery Guarantees

- ▶ Kafka guarantees that messages from a **single partition** are delivered to a consumer **in order**.



Delivery Guarantees

- ▶ Kafka guarantees that messages from a **single partition** are delivered to a consumer **in order**.
- ▶ There is **no guarantee** on the ordering of messages coming from **different partitions**.



Delivery Guarantees

- ▶ Kafka guarantees that messages from a **single partition** are delivered to a consumer **in order**.
- ▶ There is **no guarantee** on the ordering of messages coming from **different partitions**.
- ▶ Kafka only guarantees **at-least-once** delivery.



Data Stream Processing

Streaming Data

- ▶ Data stream is unbound data, which is broken into a sequence of individual tuples.
- ▶ A data tuple is the atomic data item in a data stream.
- ▶ Can be structured, semi-structured, and unstructured.



Streaming Data Processing Design Points

- ▶ Continuous vs. micro-batch processing
- ▶ Record-at-a-Time vs. declarative APIs
- ▶ Event time vs. processing time
- ▶ Windowing



Streaming Data Processing Design Points

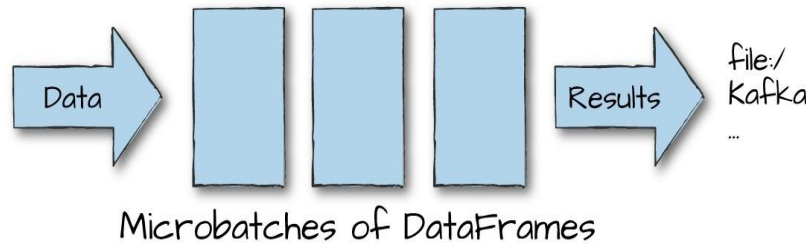
- ▶ Continuous vs. micro-batch processing
- ▶ Record-at-a-Time vs. declarative APIs
- ▶ Event time vs. processing time
- ▶ Windowing



Streaming Data Processing Patterns

► Micro-batch systems

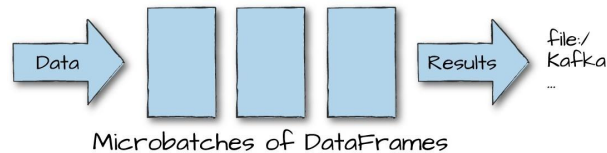
- Batch engines
- Slicing up the unbounded data into a sets of bounded data, then process each batch.



Streaming Data Processing Patterns

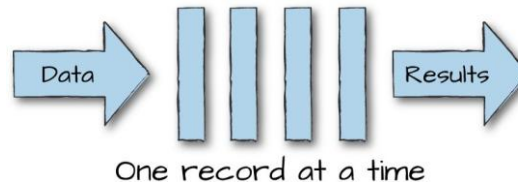
► Micro-batch systems

- Batch engines
- Slicing up the unbounded data into a sets of bounded data, then process each batch.



► Continuous processing-based systems

- Each node in the system continually listens to messages from other nodes and outputs new updates to its child nodes.



Streaming Data Processing Design Points

- ▶ Continuous vs. micro-batch processing
- ▶ Record-at-a-Time vs. declarative APIs
- ▶ Event time vs. processing time
- ▶ Windowing



Record-at-a-Time vs. Declarative APIs

► Record-at-a-Time API (e.g., Storm)

- Low-level API
- Passes each event to the application and let it react.
- Useful when applications need full control over the processing of data.
- Complicated factors, such as maintaining state, are governed by the application.



Record-at-a-Time vs. Declarative APIs

- ▶ **Record-at-a-Time** API (e.g., Storm)
 - **Low-level** API
 - Passes **each event** to the **application** and let it react.
 - Useful when applications need **full control** over the processing of data.
 - **Complicated factors**, such as maintaining state, are **governed by the application**.
- ▶ **Declarative** API (e.g., Spark streaming, Flink, Google Dataflow)
 - Applications specify **what** to compute **not how** to compute it in response to **each new event**.



Streaming Data Processing Design Points

- ▶ Continuous vs. micro-batch processing
- ▶ Record-at-a-Time vs. declarative APIs
- ▶ Event time vs. processing time
- ▶ Windowing



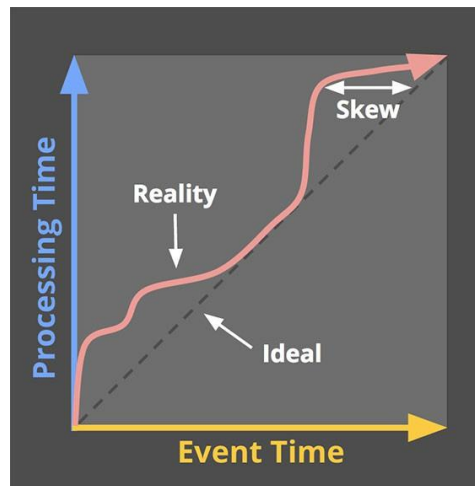
Event Time vs. Processing Time (1/2)

- ▶ **Event time**: the time at which events **actually occurred**.
 - Timestamps inserted into each record **at the source**.
- ▶ **Processing time**: the time when the record is **received at the streaming application**.



Event Time vs. Processing Time (2/2)

- Ideally, event time and processing time should be equal.
- Skew between event time and processing time.



[<https://www.oreilly.com/ideas/the-world-beyond-batch-streaming-101>]

Streaming Data Processing Design Points

- ▶ Continuous vs. micro-batch processing
- ▶ Record-at-a-Time vs. declarative APIs
- ▶ Event time vs. processing time
- ▶ Windowing



Windowing (1/2)

- ▶ **Window**: a **buffer** associated with an input port to retain previously **received tuples**.
- ▶ **Four** different windowing **management policies**.



Windowing (1/2)

- ▶ **Window**: a **buffer** associated with an input port to retain previously **received tuples**.
- ▶ **Four** different windowing **management policies**.
 - **Count-based policy**: the **maximum number** of tuples a window buffer can hold



Windowing (1/2)

- ▶ **Window**: a **buffer** associated with an input port to retain previously **received tuples**.
- ▶ **Four** different windowing **management policies**.
 - **Count-based policy**: the **maximum number** of tuples a window buffer can hold
 - **Delta-based policy**: a **delta threshold** in a tuple attribute



Windowing (1/2)

- ▶ **Window**: a **buffer** associated with an input port to retain previously **received tuples**.
- ▶ **Four** different windowing **management policies**.
 - **Count-based policy**: the **maximum number** of tuples a window buffer can hold
 - **Delta-based policy**: a **delta threshold** in a tuple attribute
 - **Punctuation-based policy**: a **punctuation** is received



Windowing (1/2)

- ▶ **Window**: a **buffer** associated with an input port to retain previously **received tuples**.
- ▶ **Four** different windowing **management policies**.
 - **Count-based policy**: the **maximum number** of tuples a window buffer can hold
 - **Delta-based policy**: a **delta threshold** in a tuple attribute
 - **Punctuation-based policy**: a **punctuation** is received
 - **Time-based policy**: based on **processing or event time** period



Windowing (2/2)

- ▶ Two types of windows: tumbling and sliding



Windowing (2/2)

- ▶ Two types of windows: tumbling and sliding
- ▶ Tumbling window: supports batch operations.
 - When the buffer fills up, all the tuples are evicted.



Windowing (2/2)

- ▶ Two types of windows: tumbling and sliding
- ▶ Tumbling window: supports batch operations.
 - When the buffer fills up, all the tuples are evicted.

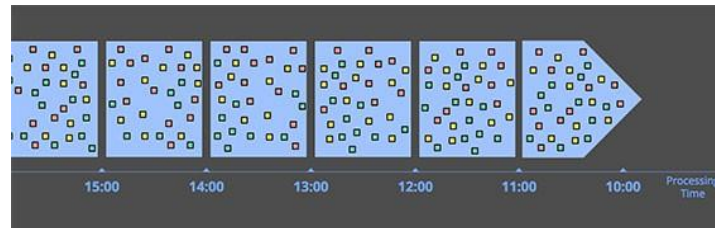


- ▶ Sliding window: supports incremental operations.
 - When the buffer fills up, older tuples are evicted.



Windowing by Processing Time

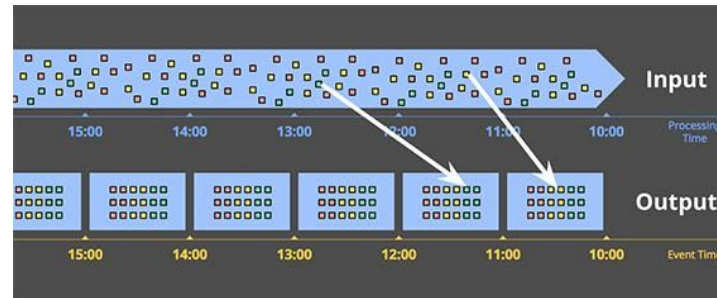
- ▶ The system **buffers up** incoming data into windows until **some amount of processing time has passed**.
- ▶ E.g., **five-minute** fixed windows



[<https://www.oreilly.com/ideas/the-world-beyond-batch-streaming-101>]

Windowing by Event Time

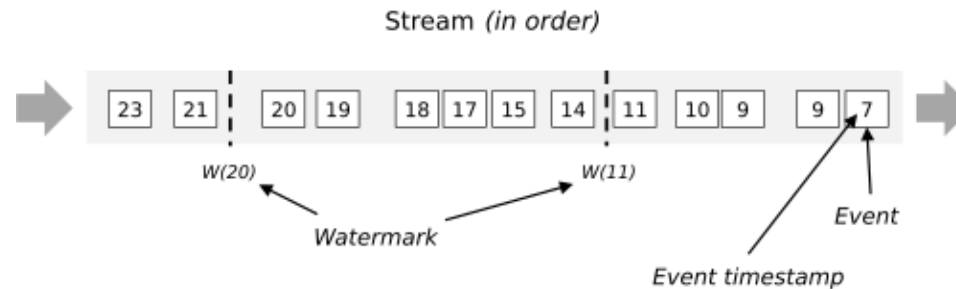
- ▶ Reflect the **times** at which **events actually happened**.
- ▶ Handling **out-of-order** events.



[<https://www.oreilly.com/ideas/the-world-beyond-batch-streaming-101>]

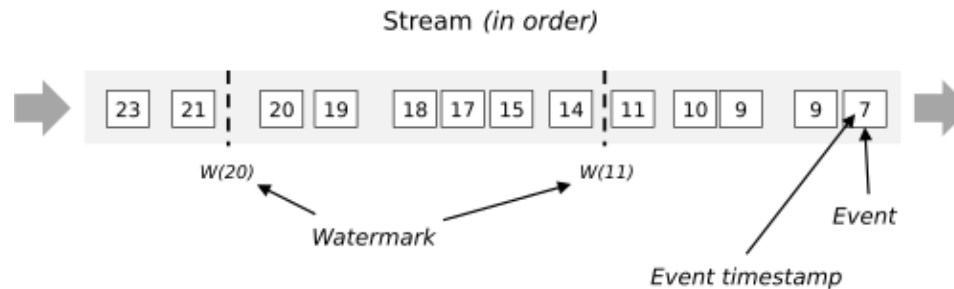
Windowing by Event Time - Watermark (1/2)

- ▶ **Watermarking** helps a stream processing system to deal with **lateness**.



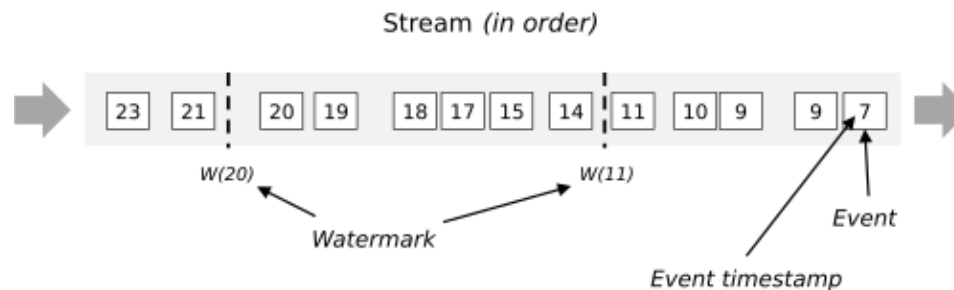
Windowing by Event Time - Watermark (1/2)

- ▶ **Watermarking** helps a stream processing system to deal with **lateness**.
- ▶ Watermarks **flow as part of the data stream** and carry a **timestamp t** .



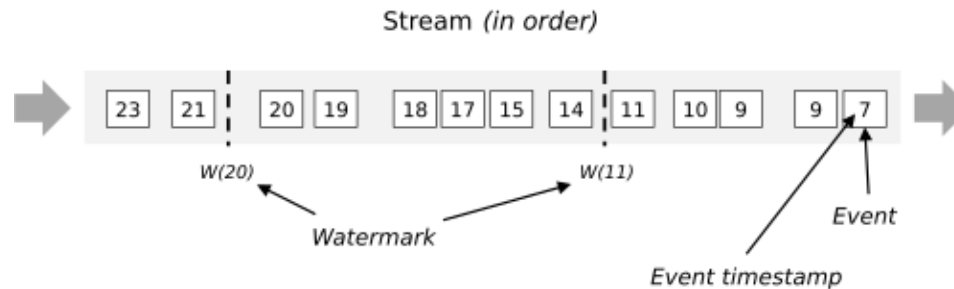
Windowing by Event Time - Watermark (1/2)

- ▶ **Watermarking** helps a stream processing system to deal with **lateness**.
- ▶ Watermarks **flow as part of the data stream** and carry a **timestamp t** .
- ▶ A watermark is a **threshold** to specify **how long the system waits** for **late events**.



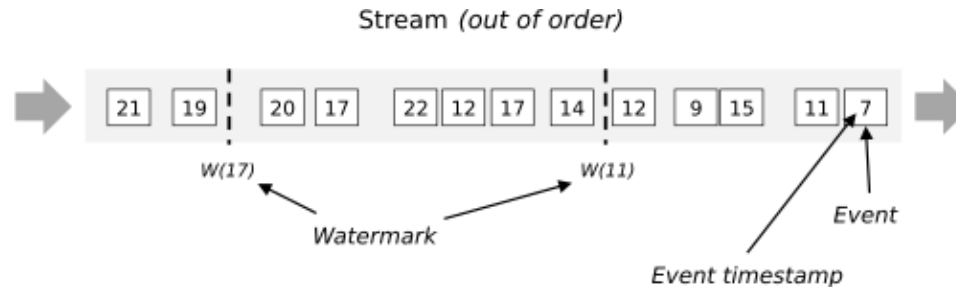
Windowing by Event Time - Watermark (1/2)

- ▶ **Watermarking** helps a stream processing system to deal with **lateness**.
- ▶ Watermarks **flow as part of the data stream** and carry a **timestamp t** .
- ▶ A watermark is a **threshold** to specify **how long the system waits** for **late events**.
- ▶ Streaming systems uses **watermarks** to **measure progress** in **event time**.



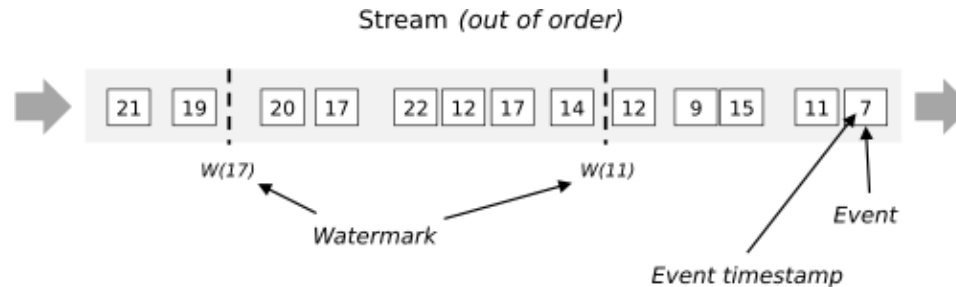
Windowing by Event Time - Watermark (2/2)

- ▶ A $W(t)$ declares that **event time** has reached time t in that stream
 - There should be **no more elements from the stream** with a timestamp $t' \leq t$.



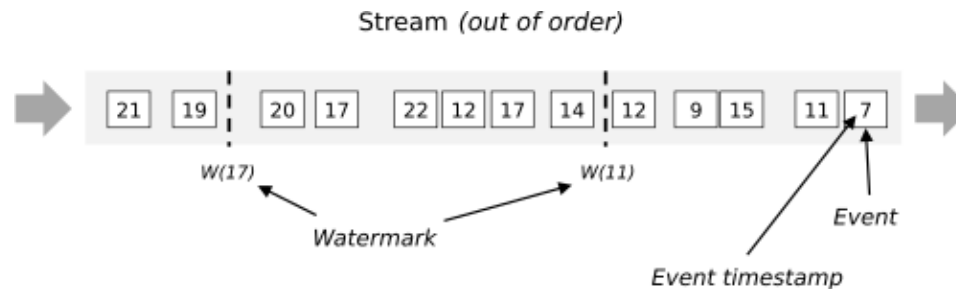
Windowing by Event Time - Watermark (2/2)

- ▶ A $W(t)$ declares that **event time** has reached time t in that stream
 - There should be **no more elements from the stream** with a timestamp $t' \leq t$.
- ▶ It is possible that certain elements will **violate the watermark condition**.
 - After the $W(t)$ has occurred, more elements with timestamp $t' \leq t$ will occur.



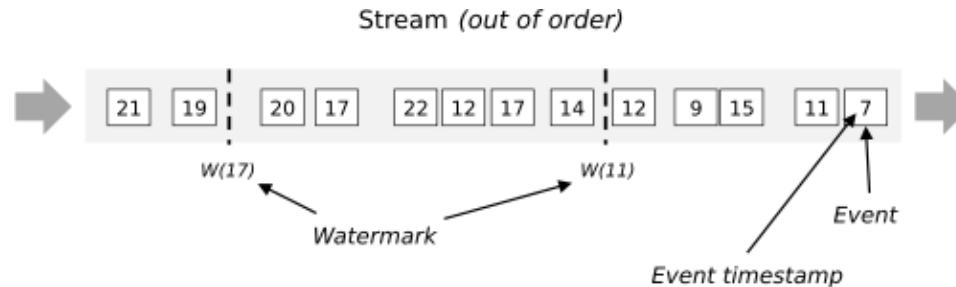
Windowing by Event Time - Watermark (2/2)

- ▶ A $W(t)$ declares that **event time** has reached time t in that stream
 - There should be **no more elements from the stream** with a timestamp $t' \leq t$.
- ▶ It is possible that certain elements will **violate the watermark condition**.
 - After the $W(t)$ has occurred, more elements with timestamp $t' \leq t$ will occur.
- ▶ If an arriving event lies **within the watermark**, it gets used to update a query.



Windowing by Event Time - Watermark (2/2)

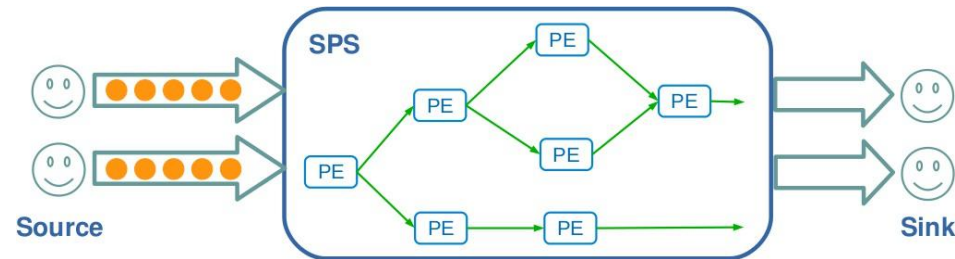
- ▶ A $W(t)$ declares that **event time** has reached time t in that stream
 - There should be **no more elements from the stream** with a timestamp $t' \leq t$.
- ▶ It is possible that certain elements will **violate the watermark condition**.
 - After the $W(t)$ has occurred, more elements with timestamp $t' \leq t$ will occur.
- ▶ If an arriving event lies **within the watermark**, it gets used to update a query.
- ▶ Streaming programs may explicitly expect some **late elements**.



Streaming Data Processing Model

Streaming Data Processing

- ▶ The tuples are processed by the application's **operators** or **processing element (PE)**.
- ▶ A **PE** is the **basic functional unit** in an application.
 - A PE processes **input** tuples, applies a **function**, and **outputs** tuples.
 - A **set of PEs** and stream **connections**, organized into a **data flow graph**.



PEs States (1/3)

- ▶ A PE can either **maintain internal state** across tuples while processing them, or process tuples **independently** of each other.
- ▶ **Stateful** vs. **stateless** tasks



PEs States (2/3)

- ▶ **Stateless** tasks: do **not maintain state** and process each tuple **independently** of **prior history**, or even from the **order** of arrival of tuples.



PEs States (2/3)

- ▶ **Stateless** tasks: do **not maintain state** and process each tuple **independently** of **prior history**, or even from the **order** of arrival of tuples.
- ▶ Easily **parallelized**.
- ▶ **No synchronization**.
- ▶ **Restart upon failures** without the need of any recovery procedure.



PEs States (3/3)

- **Stateful** tasks: involves **maintaining** information **across different tuples** to detect complex patterns.



PEs States (3/3)

- ▶ **Stateful** tasks: involves **maintaining** information **across different tuples** to detect complex patterns.
- ▶ A **PE** is usually a **synopsis** of the **tuples received so far**.
- ▶ A subset of **recent tuples** kept in a **window buffer**.



Runtime Systems

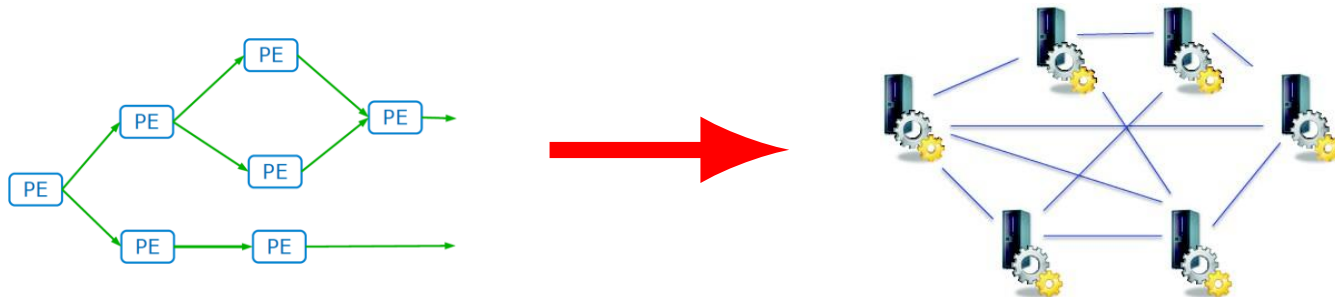
Job and Job Management

- ▶ At runtime, an **application** is represented by **one or more jobs**.
- ▶ **Jobs** are deployed as a **collection of PEs**.
- ▶ **Job management** component must **identify and track** individual **PEs**, the **jobs** they belong to, and associate them with the user that instantiated them.

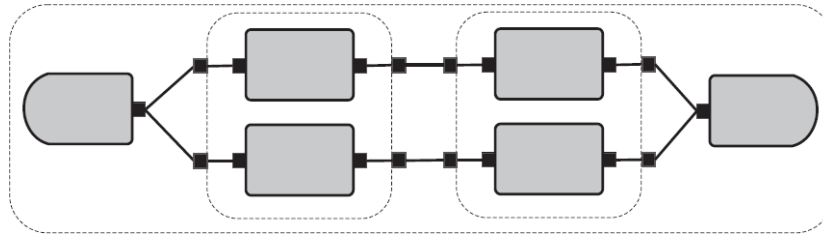


Logical Plan vs. Physical Plan (1/3)

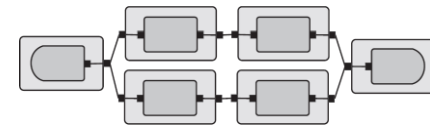
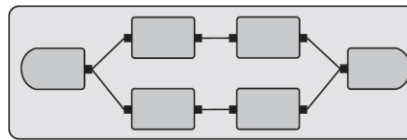
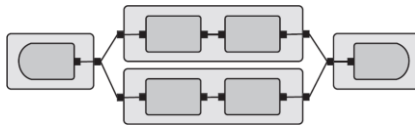
- ▶ **Logical plan**: a data flow graph, where the **vertices** correspond to **PEs**, and the **edges** to **stream connections**.
- ▶ **Physical plan**: a data flow graph, where the **vertices** correspond to OS **processes**, and the **edges** to **transport connections**.



Logical Plan vs. Physical Plan (2/3)



Logical plan



Different physical plans

Logical Plan vs. Physical Plan (3/3)

- ▶ How to map a **network of PEs** onto the **physical network of nodes**?
 - Parallelization
 - Fault tolerance
 - Optimization



Parallelization

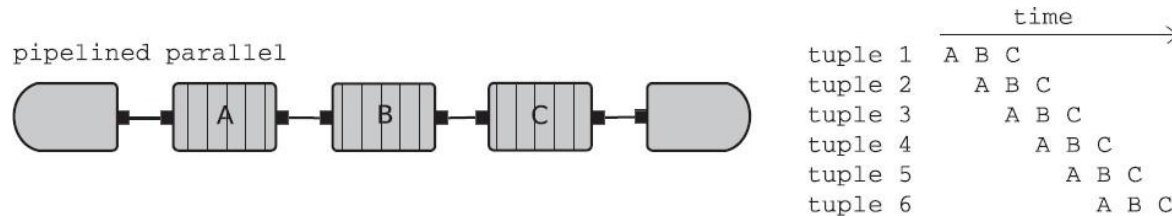
Parallelization

- ▶ How to **scale** with increasing the **number queries** and the **rate of incoming events**?
- ▶ **Three** forms of parallelisms.
 - **Pipelined** parallelism
 - **Task** parallelism
 - **Data** parallelism



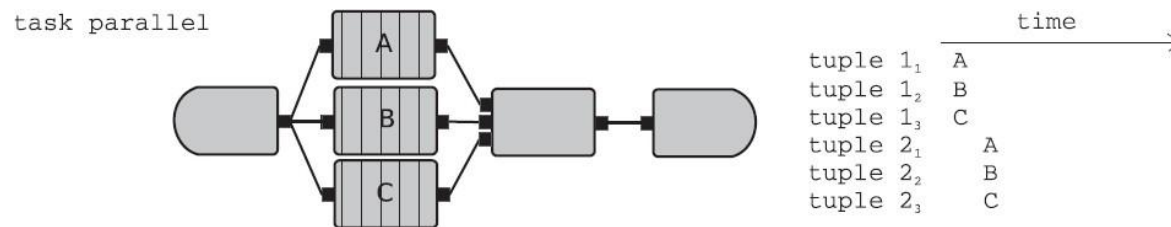
Pipelined Parallelism

- Sequential stages of a computation execute **concurrently** for **different data items**.



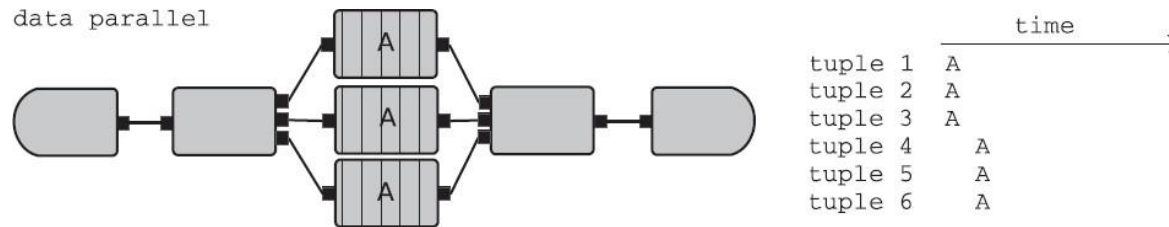
Task Parallelism

- Independent processing stages of a larger computation are executed **concurrently** on the same or distinct data items.



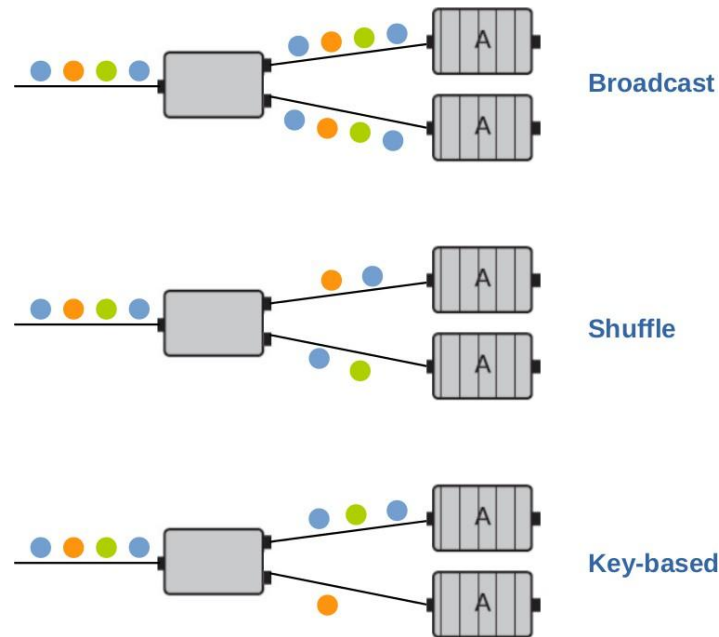
Data Parallelism (1/2)

- ▶ The same computation takes place concurrently on different data items.



Data Parallelism (2/2)

- ▶ How to **allocate** data items to each **computation** instance?



Fault Tolerance

Fault Tolerance

- ▶ The recovery methods of streaming frameworks must take:
 - **Correctness**, e.g., data loss and duplicates
 - **Performance**, e.g., low latency



Delivery Guarantees

- ▶ At-least-once: might appear many times
- ▶ Exactly-once: is consumed just once



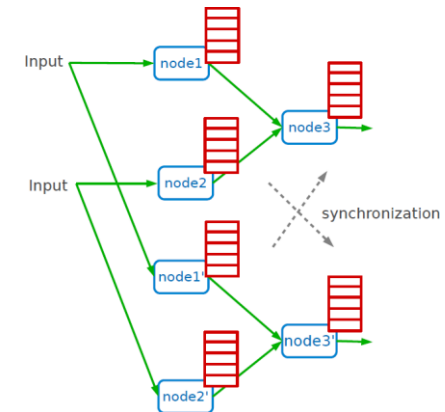
Recovery Methods

- ▶ Active backup
- ▶ Passive backup
- ▶ Upstream backup



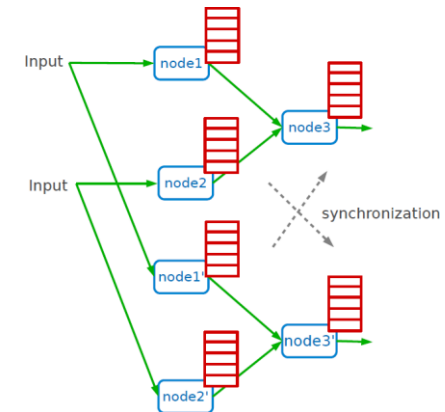
Recovery Methods - Active Backup

- ▶ Each processing node has an associated **backup node**.
- ▶ **Both** primary and backup nodes are given the **same** input.



Recovery Methods - Active Backup

- ▶ Each processing node has an associated **backup node**.
- ▶ **Both** primary and backup nodes are given the **same** input.
- ▶ If the **primary** fails, the **backup** takes over by **sending the logged tuples** to all downstream neighbors and then continuing its processing.



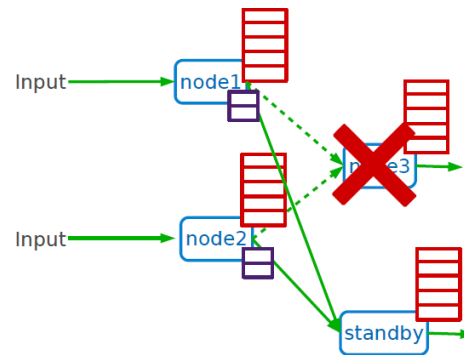
Recovery Methods - Passive Backup

- ▶ Periodically check-points processing state to a shared storage.
- ▶ The backup node takes over from the latest checkpoint when the primary fails.



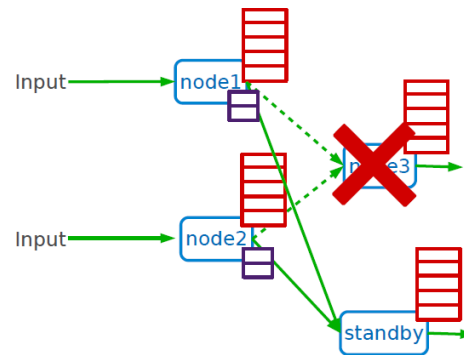
Recovery Methods - Upstream Backup

- ▶ Upstream nodes store the tuples until the downstream nodes acknowledge them.



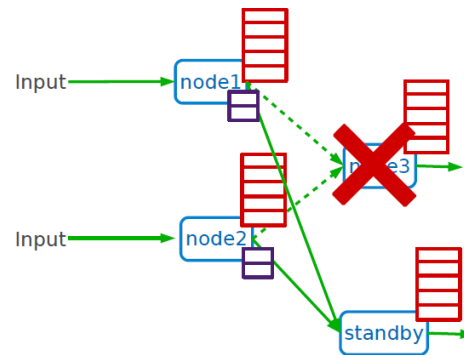
Recovery Methods - Upstream Backup

- ▶ Upstream nodes store the tuples until the downstream nodes acknowledge them.
- ▶ If a node fails, an empty node rebuilds the latest state of the failed primary from the logs kept at the upstream server.



Recovery Methods - Upstream Backup

- ▶ Upstream nodes store the tuples until the downstream nodes acknowledge them.
- ▶ If a node fails, an empty node rebuilds the latest state of the failed primary from the logs kept at the upstream server.
- ▶ There is no backup node in this model.



Summary

- ▶ Messaging system and partitioned logs
- ▶ Decoupling producers and consumers
- ▶ Kafka: distributed, topic oriented, partitioned, replicated log service
- ▶ Logs, topics, partition
- ▶ Kafka architecture: producer, consumer, broker, coordinator



Summary

- ▶ SPS vs. DBMS
- ▶ Data stream, unbounded data, tuples
- ▶ Event-time vs. processing time
- ▶ Micro-batch vs. continuous processing (windowing)
- ▶ PEs and dataflow
- ▶ Stateless vs. Stateful PEs
- ▶ SPS runtime: parallelization, fault-tolerance

